$_

100+ LINUX COMMANDS

FOR SCRIPT KIDDIES

Linux commands encompass a broad range of tasks that allow users to interact with the operating system via the terminal. Topics include file and directory management, system monitoring, user and permissions management, process control, and network operations.

bash | terminal | DevOps | sysadmin | automation | scripting | Linux | CLI



WRITTEN BY

Dr. Ranjith Shepur

DevOps & DevSecOps Expert

Satellite | ERP | CRM | Banking | E-Commerce

100+	6+	2026
Commands	Domains	Edition

FIRST EDITION • 2026 • CREATIVE COMMONS CC BY

Free to Share, Adapt & Distribute with Attribution

Copyright © 2026 by Ranjith Shepur

Licensed under Creative Commons Attribution [CC BY] International License

DevOps & DevSecOps | Satellite | ERP | CRM | Banking | E-Commerce

COPYRIGHT & LICENSE

Copyright © 2026 by Ranjith Shepur

Creative Commons Attribution [CC BY] International License

This book is licensed under a Creative Commons Attribution [CC BY] International License. The Creative Commons Attribution License allows others to share, copy, distribute, and transmit the work as long as they give appropriate credit to the original author and indicate if any changes were made to the work. The license also allows for commercial use of the work.

By choosing to release this book under a Creative Commons license, the author is allowing readers to share and distribute the book while still retaining some control over how it is used. This is because the license requires attribution and may include other restrictions, depending on the type of license chosen.

Readers are free to use the book in various ways, including adapting it, translating it, or even creating a new work based on it. However, they must still give credit to the original author and comply with any other restrictions set forth in the license.

It's important to note that while Creative Commons licenses are a useful tool for sharing and disseminating creative works, they are not a substitute for copyright law. Copyright law still applies to works licensed under a Creative Commons license, and authors should seek legal advice to ensure that their rights are protected.

In addition, the author of this book retains all other rights not specifically granted by the Creative Commons license, such as the right to create derivative works or to license the work to others under different terms.

ABOUT THE AUTHOR



Dr. Ranjith Shepur

DevOps & DevSecOps Expert | Author | Technology Leader

Dr. Ranjith Shepur is a seasoned technology professional with vast experience in DevOps and DevSecOps, bringing together deep expertise in software engineering, security, and cloud infrastructure. With a distinguished career spanning multiple domains, Dr. Shepur has contributed to mission-critical projects across diverse industries.

Key Areas of Expertise

- DevOps & DevSecOps Engineering — architecting CI/CD pipelines, container orchestration, and automated security workflows
- Satellite Project Development — contributing to high-reliability systems engineering and mission-critical software platforms
- Enterprise Resource Planning (ERP) — designing and implementing large-scale ERP solutions for complex business processes
- Customer Relationship Management (CRM) — building CRM systems that enhance customer engagement and retention
- E-Commerce Platforms — developing scalable, secure online retail solutions from architecture to deployment
- Banking Software & SDLC — delivering secure, compliant software solutions across the full Software Development Life Cycle for banking institutions

Dr. Shepur's hands-on experience across satellite systems, ERP, CRM, e-commerce, and banking software has given him a unique perspective on how Linux commands and automation are applied

in real-world, production-grade environments. This book is a distillation of those practical insights, designed to empower the next generation of Linux practitioners.

WHY YOU NEED THIS BOOK

& HOW IT WILL HELP YOU

Why This Book?

In a world where Linux powers over 90% of cloud servers, the majority of supercomputers, and the foundation of Android devices, understanding Linux commands is no longer optional — it's essential. Yet many learners find the command line intimidating, scattered, and hard to master without a structured resource.

This book bridges that gap. Curated from years of real-world DevOps and DevSecOps practice, it provides 100+ essential Linux commands organized logically, with clear explanations and practical context. Whether you're a student, developer, sysadmin, or security professional, this book is your shortcut from confusion to competence.

How This Book Will Help You

▶️ For Beginners

Start from zero and build a solid foundation. Each command is explained in plain language with real examples, so you never feel lost or overwhelmed.

▶️ For Developers

Automate repetitive tasks, manage files and processes, and interact with servers confidently — all skills that make you a more effective and versatile engineer.

▶️ For System Administrators

Manage users, monitor system health, control processes, and configure networks with precision. The commands in this book are the daily toolkit of every great sysadmin.

▶️ For Security Professionals

Understand the underlying commands that power penetration testing, log analysis, and system hardening. A strong Linux foundation is the cornerstone of any DevSecOps practice.

▶️ For Students & Job Seekers

Linux command proficiency is tested in virtually every technical interview for cloud, DevOps, and backend roles. This book gives you the edge you need to stand out.

"Mastery of Linux commands enhances system efficiency and boosts productivity for users and administrators alike."

INTRODUCTION

Master Linux Commands: Your Essential Guide to Navigating, Managing, and Automating with Power

Welcome to the world of Linux, a powerful and versatile operating system that has become the backbone of modern technology. From powering web servers and embedded systems to enabling advanced scientific research and personal computing, Linux plays a crucial role in today's digital landscape. At the heart of Linux lies a rich set of command-line tools and utilities that provide unparalleled control and flexibility for managing and interacting with your system.

This book is designed to be your comprehensive guide to mastering Linux commands. Whether you're a seasoned IT professional, a developer, or someone new to the Linux environment, you'll find valuable insights and practical examples to enhance your command-line skills. Our goal is to demystify the Linux command line and equip you with the knowledge needed to efficiently navigate, manage, and automate tasks on your Linux system.

We'll start with the basics, covering fundamental commands and concepts to help you get comfortable with the command line interface. As you progress, you'll delve into more advanced topics, including file management, system monitoring, text processing, and scripting. Each chapter is designed to build on the previous one, with hands-on examples and real-world scenarios to reinforce your learning.

By the end of this book, you'll have a solid understanding of Linux commands and be able to harness their power to streamline your workflow, troubleshoot issues, and perform complex tasks with ease. Let's embark on this journey into the command line and unlock the full potential of your Linux system!

What You Will Learn

📁 **File & Directory Management:** Navigate, create, move, copy, delete, and manage files and directories with precision.

💻 **System Monitoring:** Track system performance, memory usage, CPU activity, and disk space in real time.

🔒 **User & Permissions Management:** Manage user accounts, groups, and file permissions for secure system operation.

⚙️ **Process Control:** List, monitor, pause, resume, and terminate processes running on your system.

🌐 **Network Operations:** Configure network interfaces, test connectivity, transfer files, and diagnose network issues.

🤖 **Automation & Scripting:** Combine commands into powerful shell scripts to automate repetitive tasks and workflows.

NOTE !

Linux Command Syntax & Special Characters

In Linux, commands are tools used to perform tasks — whether manipulating files, managing processes, or configuring the system. Command usage often involves understanding basic syntax, options (flags), arguments, and special characters that modify or extend functionality.

command *executable* **[OPTIONS]** *flags / switches* **[ARGUMENTS]** *files, dirs, input*

command
The executable or tool to run (e.g., ls, cat, cp)

OPTIONS
Optional flags that modify behaviour (e.g., -l, --all,

ARGUMENTS
The target/subject: file names, directories, or other i

Special Characters (Signs) & Their Meaning

SIGN	MEANING	EXAMPLE
`>`	Redirects stdout to a file (overwrites if exists).	`echo "text" > file.txt`
`>>`	Appends stdout to the end of a file.	`echo "text" >> file.txt`
`<`	Redirects stdin from a file.	`command < inputfile.txt`
`&`	Runs a command in the background.	`command &`
`&&`	AND: 2nd command runs only if 1st succeeds.	`command1 && command2`
`*`	Wildcard: matches any sequence of characters.	`ls *.txt`
`?`	Matches any single character in a filename.	`ls file?.txt`
`~`	Represents the current user's home directory.	`cd ~`
`..`	Refers to the parent directory (one level up).	`cd ..`
`&>`	Redirects both stdout and stderr to a file.	`command &> file.txt`
`;`	Command separator: runs multiple commands in sequence.	`command1 ; command2`
`-`	Indicates an option or flag.	`ls -l`
`--`	Long-form options for commands.	`ls --all or rm --force`

Creative Commons Attribution [CC BY]

Table of Contents

Linux Commands Reference — Part 1 of 2

Commands at a Glance

1	ls	10	46	mktemp	55
2	chmod	11	47	cd	56
3	chown	12	48	netstat	57
4	chgrp	13	49	rsh	58
5	cmp	14	50	ssh	59
6	comm	15	51	nmap	60
7	cp	16	52	ifconfig	61
8	crypt	17	53	ping	62
9	diff	18	54	ip	63
10	file	19	55	iptables	64
11	grep	20	56	dig	65
12	gzip	21	57	clear	66
13	ln	22	58	env	67
14	lsof	23	59	exit	68
15	mkdir	24	60	login	69
16	mv	25	61	history	70
17	pwd	26	62	sleep	71
18	quota	27	63	su	72
19	rm	28	64	sudo	73
20	rmdir	29	65	nc	74
21	stat	30	66	nslookup	75
22	sync	31	67	wget	76
23	sort	32	68	traceroute	77
24	tar	33	69	ss	78
25	tee	34	70	mtr	79
26	gunzip	35	71	ifdown	80
27	umask	36	72	ifup	81
28	uncompress	37	73	curl	82
29	awk	38	74	clock	83
30	split	39	75	free	84
31	paste	40	76	df	85
32	cut	41	77	du	86
33	basename	42	78	crontab	87
34	uuencode	43	79	finger	88
35	uudecode	44	80	last	89
36	base32	45	81	lpq	90
37	base64	46	82	man	91
38	cfdisk	47	83	path	92
39	chattr	48	84	printenv	93
40	gawk	49	85	ps	94
41	getfacl	50	86	set	95
42	link	51	87	ionice	96
43	cpio	52	88	time	97
44	locate	53	89	uptime	98
45	csplit	54	90	w	99

Table of Contents

Linux Commands Reference — Part 2 of 2

Commands at a Glance

91	who	100
92	whois	101
93	whoami	102
94	iostat	103
95	uname	104
96	useradd	105
97	usermod	106
98	vmstat	107
99	shutdown	108
100	service	109
101	userdel	110
102	pstree	111
103	ncdu	112
104	hostname	113
105	etcdctl	114
106	groupadd	115
107	access	116
108	kill	117
109	bg	118
110	fg	119
111	jobs	120
112	top	121
113	killall	122
114	fuser	123
115	pkill	124
116	wait	125
117	cat	126
118	fold	127
119	head	128
120	lpr	129
121	lprm	130
122	more	131
123	less	132
124	page	133
125	pr	134
126	tail	135
127	zcat	136
128	xv	137
129	gv	138
130	xpdf	139
131	logout	140
132	passwd	141
133	rlogin	142
134	slogin	143
135	emacs	144
136	pico	145
137	sed	146
138	vim	147
139	ftp	148
140	rsync	149
141	scp	150
142	apropos	151
143	find	152
144	info	153
145	whatis	154
146	whereis	155
147	pine	156
148	mesg	157
149	talk	158
150	write	159
151	strace	160
152	pv	161
153	groupdel	162
154	groupmod	163
155	groups	164
156	ethtool	165
157	apt	166
158	rpm	167
159	yum	168
160	hostnamectl	169
161	reboot	170
162	chsh	171
163	compress	172
164	dpkg	173
165	dosfsck	174
166	ftpd	175
167	gpasswd	176
168	hexdump	177

\$ ls

ls [OPTION]... [FILE]...

Purpose

The ls command lists directory contents, showing files and directories. It displays file properties including permissions, ownership, size, and modification date, helping users navigate the filesystem effectively.

Syntax

```
ls [OPTION]... [FILE]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-a`	List all files, including hidden files (starting with a dot)
`-l`	Long listing format with permissions, ownership, size, date
`-h`	Human-readable sizes (KB, MB, GB) — use with -l
`-r`	Reverse the sort order of the listing
`-t`	Sort by modification time, newest first
`-R`	Recursively list all subdirectories
`-S`	Sort by file size, largest first
`-1`	List one file per line
`--color`	Colorize output to distinguish file types

Examples

1. **`$ ls`**
   List files in the current directory
2. **`$ ls -la`**
   List all files including hidden with details
3. **`$ ls -lh /var/log`**
   List /var/log files with human-readable sizes
4. **`$ ls -lt`**
   List files sorted by modification time
5. **`$ ls -R /etc`**
   Recursively list all files under /etc

\$chmod

chmod [OPTION]... MODE FILE...

Purpose

The chmod command changes file or directory permissions in Linux. It controls read, write, and execute access for the owner, group, and others, using symbolic or octal notation to set access levels.

Syntax

```
chmod [OPTION]... MODE FILE...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-R`	Apply permission changes recursively to directories
`-v`	Verbose: output a diagnostic for every file processed
`-c`	Like verbose, but only reports when a change is made
`u+x`	Add execute permission for the file owner
`g-w`	Remove write permission from the group
`o=r`	Set others' permission to read-only
`755`	Octal: owner rwx, group r-x, others r-x
`644`	Octal: owner rw-, group r--, others r--

Examples

1. **`$ chmod 755 script.sh`**
   Set script executable by owner, readable by all

2. **`$ chmod +x deploy.sh`**
   Add execute permission for everyone

3. **`$ chmod -R 644 /var/www`**
   Recursively set 644 on web files

4. **`$ chmod u+x,g-w file.txt`**
   Add exec for owner, remove write from group

5. **`$ chmod o= file.txt`**
   Remove all permissions for others

\$chown

chown [OPTION]... OWNER[:GROUP] FILE...

Purpose

The chown command changes the ownership of files and directories. It allows administrators to transfer file ownership to a different user and/or group, which is essential for system administration and security management.

Syntax

```
chown [OPTION]... OWNER[:GROUP] FILE...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-R`	Recursively change ownership of directories and their contents
`-v`	Verbose output for each file processed
`-c`	Report only when a change is actually made
`--from=OWNER`	Change only if current owner matches OWNER
`OWNER:GROUP`	Set both user and group ownership simultaneously
`OWNER:`	Change owner; retain current group

Examples

1. `$ chown alice file.txt`
   Change owner of file.txt to alice
2. `$ chown alice:developers file.txt`
   Change owner to alice and group to developers
3. `$ chown -R www-data /var/www`
   Recursively set web server ownership
4. `$ chown :staff report.pdf`
   Change only group to staff
5. `$ chown --from=bob:bob -R alice /home/shared`
   Change ownership only if currently owned by bob

\$chgrp

chgrp [OPTION]... GROUP FILE...

Purpose

The chgrp command changes the group ownership of files and directories. It is used to assign a file to a specific group, enabling group-level access control and permissions management.

Syntax

```
chgrp [OPTION]... GROUP FILE...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-R`	Recursively change group of directories and contents
`-v`	Display a message for each processed file
`-c`	Report only when a change is made
`--reference=FILE`	Use the group of a reference file
`-h`	Change the group of symbolic links instead of their targets

Examples

1. `$ chgrp staff report.txt`
   Change group of report.txt to staff
2. `$ chgrp -R developers /srv/app`
   Recursively assign group developers
3. `$ chgrp --reference=ref.txt target.txt`
   Use group from ref.txt for target.txt
4. `$ chgrp -v audio /dev/snd/*`
   Change group and show verbose output
5. `$ chgrp -R www-data /var/www/html`
   Set web directory group to www-data

\$cmp

cmp [OPTION]... FILE1 FILE2

Purpose

The cmp command compares two files byte by byte. It reports the first position where the files differ, or silently exits if they are identical. It is useful for detecting differences in binary files where diff may not be suitable.

Syntax

```
cmp [OPTION]... FILE1 FILE2
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-l`	Print byte number and differing bytes for all differences
`-s`	Silent mode: return exit status only (0=same, 1=differ)
`-i N`	Skip first N bytes of both files before comparing
`-n N`	Compare at most N bytes
`-b`	Print differing bytes as characters

Examples

1. `$ cmp file1.txt file2.txt`
   Compare two text files byte by byte
2. `$ cmp -s image1.png image2.png`
   Silently compare; check exit code
3. `$ cmp -l bin1 bin2`
   List all differing byte positions
4. `$ cmp -i 512 disk1.img disk2.img`
   Skip first 512 bytes then compare
5. `$ cmp -n 1024 a.bin b.bin`
   Compare only the first 1024 bytes

\$comm

comm [OPTION]... FILE1 FILE2

Purpose

The comm command compares two sorted files line by line and outputs three columns: lines unique to file1, lines unique to file2, and lines common to both. It is useful for finding set differences and intersections between sorted lists.

Syntax

```
comm [OPTION]... FILE1 FILE2
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-1`	Suppress column 1 (lines only in file1)
`-2`	Suppress column 2 (lines only in file2)
`-3`	Suppress column 3 (lines common to both)
`--nocheck-order`	Do not check that the input is sorted
`--output-delimiter=STR`	Use STR as column delimiter

Examples

1. `$ comm sorted1.txt sorted2.txt`
   Show all three columns of comparison
2. `$ comm -12 file1.txt file2.txt`
   Show only lines common to both files
3. `$ comm -23 file1.txt file2.txt`
   Show lines only in file1
4. `$ comm -13 file1.txt file2.txt`
   Show lines only in file2
5. `$ comm -3 list1 list2`
   Hide common lines, show unique from each

\$cp

cp [OPTION]... SOURCE DEST

Purpose

The cp command copies files and directories from one location to another. It supports recursive copying of directories, preserving file attributes, creating symbolic links instead of copies, and handling multiple source files.

Syntax

```
cp [OPTION]... SOURCE DEST
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-r / -R`	Copy directories recursively
`-p`	Preserve file attributes: mode, ownership, timestamps
`-i`	Interactive: prompt before overwriting
`-u`	Copy only when source is newer than destination
`-v`	Verbose: show files being copied
`-a`	Archive mode: preserve all attributes and links
`-b`	Create backup of existing destination files
`-n`	No-clobber: do not overwrite existing files

Examples

1. `$ cp file.txt /backup/`
   Copy file.txt to /backup/ directory
2. `$ cp -r /src/project /dst/project`
   Recursively copy entire project directory
3. `$ cp -p config.conf /etc/app/`
   Copy preserving permissions and timestamps
4. `$ cp -i *.log /var/archive/`
   Interactively copy log files
5. `$ cp -u src/*.py dest/`
   Copy only newer Python files to dest

\$crypt

crypt [PASSWORD] < inputfile > outputfile

Purpose

The crypt command encrypts or decrypts files using a password. It provides basic symmetric encryption for protecting file contents. Note: crypt is considered legacy; modern alternatives like gpg or openssl are preferred for strong encryption.

Syntax

```
crypt [PASSWORD] < inputfile > outputfile
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`< file`	Read input from file via stdin redirection
`PASSWORD`	Encryption key provided as argument (insecure) or interactively
`-k`	Read key from the CRYPTKEY environment variable

Examples

1. `$ crypt < secret.txt > secret.enc`
   Encrypt secret.txt interactively
2. `$ crypt mypassword < file.txt > file.enc`
   Encrypt with inline password
3. `$ crypt mypassword < file.enc > file.txt`
   Decrypt previously encrypted file
4. `$ crypt < enc.txt > dec.txt`
   Decrypt using interactive password prompt
5. `$ CRYPTKEY=pass crypt -k < in.txt > out.txt`
   Use environment variable for key

\$diff

diff [OPTION]... FILE1 FILE2

Purpose

The diff command compares two files line by line and outputs the differences between them. It is widely used to create patches, review changes, and understand what has been modified between two versions of a file or directory.

Syntax

```
diff [OPTION]... FILE1 FILE2
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-u`	Unified format: shows context lines around changes
`-r`	Recursively compare directories
`-i`	Ignore case differences
`-w`	Ignore all whitespace differences
`-q`	Brief mode: report only whether files differ
`-b`	Ignore changes in amount of whitespace
`-c`	Context format showing surrounding lines
`--color`	Colorize the output (GNU diff)

Examples

1. `$ diff old.txt new.txt`
   Compare two files and show differences
2. `$ diff -u v1.py v2.py`
   Unified diff suitable for patches
3. `$ diff -r dir1/ dir2/`
   Recursively compare two directories
4. `$ diff -i file1 file2`
   Compare ignoring case differences
5. `$ diff -q /etc/hosts /backup/hosts`
   Check if files differ without showing diff

\$file

file [OPTION]... FILE...

Purpose

The file command determines the type of a file by examining its content, rather than relying on the file extension. It reads magic bytes and other file characteristics to identify formats such as text, binary, image, archive, or executable.

Syntax

```
file [OPTION]... FILE...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-i`	Output MIME type string instead of human-readable type
`-b`	Brief mode: do not prepend filenames
`-z`	Look inside compressed files
`-L`	Follow symbolic links
`-f list`	Read filenames from a list file
`--mime-encoding`	Output the MIME encoding of the file

Examples

1. `$ file document.pdf`
   Identify the type of document.pdf
2. `$ file -i image.png`
   Get MIME type of image.png
3. `$ file *`
   Identify types of all files in current directory
4. `$ file -z archive.gz`
   Identify compressed file contents
5. `$ file -b script.sh`
   Show type without filename prefix

\$grep

grep [OPTION]... PATTERN [FILE]...

Purpose

The grep command searches text files for lines matching a given pattern or regular expression. It is one of the most powerful and commonly used Unix tools for filtering output, searching logs, and processing text data.

Syntax

```
grep [OPTION]... PATTERN [FILE]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-i`	Case-insensitive matching
`-r / -R`	Recursively search directories
`-n`	Print line numbers with matching lines
`-v`	Invert match: show lines that do NOT match
`-c`	Print only count of matching lines
`-l`	Print only names of files with matches
`-E`	Extended regex (same as egrep)
`-A N`	Show N lines after each match

Examples

1. `$ grep 'error' /var/log/syslog`
   Search for 'error' in syslog
2. `$ grep -rn 'TODO' /src/`
   Recursively find TODOs with line numbers
3. `$ grep -i 'warning' app.log`
   Case-insensitive search for warnings
4. `$ grep -v '^#' config.conf`
   Show non-comment lines in config
5. `$ grep -c 'FAILED' auth.log`
   Count failed authentication attempts

\$gzip

gzip [OPTION]... FILE...

Purpose

The gzip command compresses files using the LZ77 algorithm to reduce storage space and speed up file transfers. By default it replaces the original file with a compressed .gz version, and it can also decompress .gz files.

Syntax

```
gzip [OPTION]... FILE...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-d`	Decompress (same as gunzip)
`-k`	Keep original file after compression
`-r`	Recursively compress files in a directory
`-v`	Verbose: show compression ratio
`-1 to -9`	Compression level: 1=fastest, 9=best compression
`-l`	List compression info for .gz files
`-t`	Test integrity of compressed file
`-c`	Write to stdout, keep original

Examples

1. **`$ gzip largefile.log`**
   Compress largefile.log to largefile.log.gz

2. **`$ gzip -d file.gz`**
   Decompress file.gz

3. **`$ gzip -k -9 data.csv`**
   Best compression, keep original file

4. **`$ gzip -v *.log`**
   Compress all logs with verbose output

5. **`$ gzip -l archive.gz`**
   List details of compressed archive

\$ ln

ln [OPTION]... TARGET LINK_NAME

Purpose

The ln command creates links between files — either hard links or symbolic (soft) links. Symbolic links act like shortcuts pointing to another file or directory path, while hard links create additional directory entries for the same inode.

Syntax

```
ln [OPTION]... TARGET LINK_NAME
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-s`	Create a symbolic (soft) link
`-f`	Force: remove existing destination file
`-i`	Interactive: prompt before removing destinations
`-v`	Verbose: print name of each linked file
`-r`	Create symbolic links relative to link location
`-b`	Backup existing destination files
`-n`	Treat destination as normal file if it is a symlink

Examples

1. `$ ln -s /etc/nginx/sites-available/app /etc/nginx/sites-enabled/app`
   Create symlink for nginx site
2. `$ ln -s /usr/bin/python3 /usr/local/bin/python`
   Symlink python3 as python
3. `$ ln file.txt hardlink.txt`
   Create a hard link
4. `$ ln -sf new_target existing_link`
   Force update an existing symlink
5. `$ ln -sv /data/shared /home/user/shared`
   Verbose symlink creation

\$ lsof

lsof [OPTION]... [FILE/PID/USER]

Purpose

The lsof command lists all open files in the system, including network connections, devices, and pipes. It is invaluable for troubleshooting, identifying which process has a file open, and investigating network connections.

Syntax

```
lsof [OPTION]... [FILE/PID/USER]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-p PID`	List files opened by a specific process ID
`-u USER`	List files opened by a specific user
`-i`	List all network connections
`-i :PORT`	List processes using a specific port
`-t`	Output only PIDs (useful in scripts)
`-c CMD`	List files for processes matching command name
`-d FD`	List files with specific file descriptor
`-n`	Inhibit DNS resolution for faster output

Examples

1. **`$ lsof /var/log/app.log`**
   Find which process has app.log open

2. **`$ lsof -u www-data`**
   List all files opened by www-data user

3. **`$ lsof -i :80`**
   Find process using port 80

4. **`$ lsof -i TCP:443`**
   Show processes on HTTPS port

5. **`$ lsof -p 1234`**
   List all open files for PID 1234

\$mkdir

mkdir [OPTION]... DIRECTORY...

Purpose

The mkdir command creates new directories. It can create multiple directories at once and automatically create parent directories in a path that don't yet exist. It also allows setting the permissions of newly created directories.

Syntax

```
mkdir [OPTION]... DIRECTORY...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-p`	Create parent directories as needed; no error if exists
`-m MODE`	Set directory permissions (e.g., 755)
`-v`	Verbose: print a message for each created directory
`-Z`	Set SELinux security context

Examples

1. `$ mkdir projects`
   Create a single directory named projects
2. `$ mkdir -p /var/app/logs/2026`
   Create full path, including parents
3. `$ mkdir -m 700 private`
   Create directory with restricted permissions
4. `$ mkdir -pv /opt/app/{bin,lib,conf}`
   Create multiple subdirectories with verbose output
5. `$ mkdir dir1 dir2 dir3`
   Create multiple directories at once

\$mv

mv [OPTION]... SOURCE DEST

Purpose

The mv command moves or renames files and directories. Unlike cp, it does not create a copy — the original is removed from its source location. It works atomically on the same filesystem and is used for both relocation and renaming.

Syntax

```
mv [OPTION]... SOURCE DEST
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-i`	Interactive: prompt before overwriting
`-f`	Force: do not prompt before overwriting
`-u`	Move only when source is newer than destination
`-v`	Verbose: show files being moved
`-b`	Backup existing destination files
`-n`	No-clobber: do not overwrite existing files

Examples

1. `$ mv old.txt new.txt`
   Rename old.txt to new.txt
2. `$ mv file.txt /backup/`
   Move file.txt to /backup/ directory
3. `$ mv -i *.log /archive/`
   Interactively move log files
4. `$ mv -v project/ /srv/apps/`
   Move directory with verbose output
5. `$ mv -u *.conf /etc/app/`
   Move only newer config files

\$pwd

pwd [OPTION]

Purpose

The pwd command prints the full absolute path of the current working directory. It is useful for confirming your location in the filesystem, especially when navigating deeply nested directories or writing scripts that depend on the current path.

Syntax

```
pwd [OPTION]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-L`	Logical: use PWD from environment (default); may include symlinks
`-P`	Physical: resolve all symlinks, show actual path

Examples

1. `$ pwd`
   Print current working directory
2. `$ pwd -P`
   Print physical path resolving symlinks
3. `$ echo "Working in: $(pwd)"`
   Embed current path in a message
4. `$ DIR=$(pwd) && cd /tmp && cd $DIR`
   Save and restore working directory
5. `$ ls $(pwd)`
   List files in the current directory path

\$quota

quota [OPTION]... [USER]

Purpose

The quota command displays disk usage and limits for users and groups on filesystems with quotas enabled. It helps administrators enforce storage limits and allows users to check their current disk usage against their allocated quota.

Syntax

```
quota [OPTION]... [USER]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-u`	Display user quota (default)
`-g`	Display group quota
`-v`	Verbose: show quotas on all quota-enabled filesystems
`-s`	Human-readable sizes
`-q`	Quiet: print only when over limit
`-l`	Do not resolve IP addresses for NFS

Examples

1. `$ quota`
   Show your own disk quota
2. `$ quota -u alice`
   Show quota for user alice
3. `$ quota -g developers`
   Show quota for group developers
4. `$ quota -vs`
   Verbose quota with human-readable sizes
5. `$ quota -q`
   Show only if you are over your quota

\$rm

rm [OPTION]... FILE...

Purpose

The rm command removes files and directories from the filesystem permanently. Unlike deletion in graphical interfaces, rm does not move files to a trash bin — deletion is immediate and typically irreversible without special recovery tools.

Syntax

```
rm [OPTION]... FILE...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-r / -R`	Recursively remove directories and their contents
`-f`	Force: ignore nonexistent files; no prompts
`-i`	Interactive: prompt before each removal
`-I`	Prompt once before removing more than 3 files
`-v`	Verbose: explain what is being done
`--preserve-root`	Do not remove root / (default protection)

Examples

1. `$ rm file.txt`
   Remove a single file
2. `$ rm -rf /tmp/build`
   Forcefully remove directory and all contents
3. `$ rm -i *.tmp`
   Interactively delete all .tmp files
4. `$ rm -v oldlog-*.log`
   Remove log files with verbose output
5. `$ rm -rf node_modules/`
   Remove node_modules directory

\$rmdir

rmdir [OPTION]... DIRECTORY...

Purpose

The rmdir command removes empty directories. Unlike rm -r, it will not remove directories that contain files or subdirectories — providing a safer way to clean up directory structures without accidentally deleting content.

Syntax

```
rmdir [OPTION]... DIRECTORY...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-p`	Remove DIRECTORY and its ancestors if they become empty
`-v`	Verbose: print message for each removed directory
`--ignore-fail-on-non-e`	Suppress error for non-empty directories

Examples

1. **`$ rmdir empty_dir`**
   Remove a single empty directory

2. **`$ rmdir -p a/b/c`**
   Remove c, then b, then a if each becomes empty

3. **`$ rmdir -v dir1 dir2 dir3`**
   Remove multiple directories verbosely

4. **`$ rmdir --ignore-fail-on-non-empty dir`**
   Try to remove, ignore if not empty

5. **`$ rmdir tmp_*`**
   Remove all matching empty directories

\$stat

stat [OPTION]... FILE...

Purpose

The stat command displays detailed metadata about a file or filesystem, including size, permissions, inode number, number of links, ownership, and various timestamps (access, modification, change). It is more informative than ls -l.

Syntax

```
stat [OPTION]... FILE...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-f`	Display filesystem status instead of file status
`-c FORMAT`	Use a custom output format string
`-L`	Follow symbolic links
`-t`	Terse output: print info on one line
`--printf=FORMAT`	Like -c but interpret backslash escapes

Examples

1. `$ stat file.txt`
   Show all metadata for file.txt
2. `$ stat -f /home`
   Show filesystem statistics for /home
3. `$ stat -c '%n %s %U' *.txt`
   Show name, size, and owner of all .txt files
4. `$ stat -L symlink`
   Follow symlink and show target's stats
5. `$ stat --printf='%A %U:%G %n\n' *`
   Print permissions, owner, group, name

\$sync

sync [OPTION]... [FILE]...

Purpose

The sync command flushes all pending writes from the kernel's write buffer cache to persistent storage (disk). It ensures that data written to files has been safely stored, which is important before unmounting filesystems or shutting down.

Syntax

```
sync [OPTION]... [FILE]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-d`	Sync only data (like fdatasync), not metadata
`-f`	Sync filesystem containing specified file
`--file-system`	Sync filesystems containing each FILE

Examples

1. `$ sync`
   Flush all write buffers to disk
2. `$ sync; sync; sync`
   Triple sync before unmounting
3. `$ sync /dev/sda`
   Sync specific block device
4. `$ sync -d file.db`
   Sync data only for file.db
5. `$ sync --file-system /home`
   Sync filesystem containing /home

\$sort

sort [OPTION]... [FILE]...

Purpose

The sort command sorts lines of text files alphabetically or numerically. It can sort by specific fields, reverse order, remove duplicates, and handle complex multi-key sort criteria. It is a fundamental text processing tool in shell pipelines.

Syntax

```
sort [OPTION]... [FILE]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-n`	Numeric sort (compare according to string numerical value)
`-r`	Reverse: sort in descending order
`-k N`	Sort by Nth field/column
`-t CHAR`	Use CHAR as field delimiter
`-u`	Unique: output only the first of equal lines
`-f`	Fold case: ignore case when sorting
`-h`	Human numeric sort (e.g., 2K, 1G)
`-M`	Month sort: JAN < FEB < ... < DEC

Examples

1. `$ sort names.txt`
   Sort file alphabetically
2. `$ sort -n sizes.txt`
   Sort file numerically
3. `$ sort -rn scores.csv`
   Reverse numeric sort
4. `$ sort -t',' -k2 data.csv`
   Sort CSV by second column
5. `$ sort -u duplicates.txt`
   Sort and remove duplicate lines

\$tar

tar [OPTION]... [FILE]...

Purpose

The tar command creates and extracts archive files, commonly used for packaging multiple files into a single archive for backup or distribution. Combined with compression tools like gzip or bzip2, it produces .tar.gz or .tar.bz2 files.

Syntax

```
tar [OPTION]... [FILE]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-c`	Create a new archive
`-x`	Extract files from archive
`-v`	Verbose: list files being processed
`-f FILE`	Specify archive filename
`-z`	Compress/decompress with gzip (.gz)
`-j`	Compress/decompress with bzip2 (.bz2)
`-t`	List contents of archive without extracting
`-C DIR`	Extract files to specified directory

Examples

1. `$ tar -czf backup.tar.gz /var/www`
   Create gzipped archive of /var/www
2. `$ tar -xzf backup.tar.gz`
   Extract a .tar.gz archive
3. `$ tar -tvf archive.tar`
   List archive contents
4. `$ tar -xzf data.tar.gz -C /opt`
   Extract to /opt directory
5. `$ tar -czf - /src | ssh host 'tar -xzf - -C /dst'`
   Pipe archive over SSH

\$tee

tee [OPTION]... [FILE]...

Purpose

The tee command reads from standard input and writes to both standard output and one or more files simultaneously. It is used in pipelines to save intermediate output to a file while still passing it along to the next command.

Syntax

```
tee [OPTION]... [FILE]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-a`	Append to files rather than overwriting
`-i`	Ignore interrupt signals

Examples

1. `$ ls -la | tee listing.txt`
   List files and save to listing.txt
2. `$ echo 'update' | sudo tee /etc/motd`
   Write to a root-owned file via sudo
3. `$ command | tee output.log | grep error`
   Save all output and filter for errors
4. `$ make 2>&1 | tee build.log`
   Capture both stdout and stderr during build
5. `$ cat data.csv | tee backup.csv | sort`
   Backup data while processing it

\$gunzip

gunzip [OPTION]... FILE...

Purpose

The gunzip command decompresses files compressed with gzip (files with .gz, .z, .Z, .tgz, or .taz extensions). It is essentially equivalent to gzip -d and restores the original file, removing the compressed version by default.

Syntax

```
gunzip [OPTION]... FILE...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-k`	Keep original compressed file after decompression
`-v`	Verbose: show compression ratios
`-c`	Write output to stdout; keep original file
`-r`	Recursively decompress files in directory
`-t`	Test integrity of compressed file
`-l`	List compression statistics

Examples

1. `$ gunzip archive.gz`
   Decompress archive.gz
2. `$ gunzip -k data.csv.gz`
   Decompress but keep the .gz file
3. `$ gunzip -v *.gz`
   Decompress all .gz files verbosely
4. `$ gunzip -c file.gz > file.txt`
   Decompress to stdout, redirect to file
5. `$ gunzip -t file.gz`
   Test integrity of compressed file

\$umask

umask [OPTION]... [MODE]

Purpose

The umask command sets or displays the default permission mask applied when new files and directories are created. The mask bits specify which permissions are removed by default; a umask of 022 results in new files getting 644 permissions.

Syntax

```
umask [OPTION]... [MODE]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-S`	Display mask in symbolic notation (e.g., u=rwx,g=rx,o=rx)
`-p`	Output in a form that can be reused as input

Examples

1. **`$ umask`**
   Display current umask value
2. **`$ umask -S`**
   Display umask in symbolic notation
3. **`$ umask 022`**
   Set umask to 022 (new files get 644)
4. **`$ umask 077`**
   Restrict new files to owner only (600)
5. **`$ umask 002`**
   Allow group write on new files (664)

\$uncompress

uncompress [OPTION]... FILE...

Purpose

The uncompress command decompresses files that were compressed with the compress command (files with .Z extension). It is a legacy decompression tool; modern systems typically use gunzip or other utilities for compressed files.

Syntax

```
uncompress [OPTION]... FILE...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-c`	Write output to stdout and keep original
`-v`	Verbose: write decompression ratio to stderr
`-f`	Force decompression even if output file exists

Examples

1. **`$ uncompress file.Z`**
   Decompress a .Z compressed file

2. **`$ uncompress -v data.Z`**
   Decompress with verbose output

3. **`$ uncompress -c file.Z > file.txt`**
   Decompress to stdout

4. **`$ uncompress -f old.Z`**
   Force decompression

5. **`$ uncompress archive1.Z archive2.Z`**
   Decompress multiple .Z files

\$awk

awk [OPTION]... 'PROGRAM' [FILE]...

Purpose

The awk command is a powerful text processing tool and programming language that processes files line by line. It is ideal for extracting fields, performing calculations, formatting output, and generating reports from structured text data.

Syntax

```
awk [OPTION]... 'PROGRAM' [FILE]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-F SEP`	Set field separator (e.g., -F',' for CSV)
`-v VAR=VAL`	Assign a variable before execution
`-f FILE`	Read awk program from a file
`NR`	Built-in: current line (record) number
`NF`	Built-in: number of fields in current line
`$1,$2...`	Field variables: $1=first field, $2=second, etc.

Examples

1. `$ awk '{print $1}' file.txt`
   Print first field of each line
2. `$ awk -F',' '{print $2}' data.csv`
   Print second column of CSV
3. `$ awk 'NR==5' file.txt`
   Print the 5th line
4. `$ awk '{sum+=$1} END{print sum}' nums.txt`
   Sum all numbers in first column
5. `$ awk '/error/{print NR": "$0}' log`
   Print numbered error lines

\$split

split [OPTION]... [INPUT [PREFIX]]

Purpose

The split command divides a large file into smaller pieces. It is useful when transferring large files over networks with size limits, processing data in chunks, or breaking log files into manageable parts. Files are named with sequential suffixes.

Syntax

```
split [OPTION]... [INPUT [PREFIX]]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-l N`	Split into files of N lines each
`-b SIZE`	Split into files of SIZE bytes (e.g., 10M, 500K)
`-n N`	Split into N files of roughly equal size
`-d`	Use numeric suffixes instead of alphabetic
`--suffix-length=N`	Use suffixes of length N
`-a N`	Use N characters for suffix (default 2)

Examples

1. `$ split -l 1000 large.txt chunk_`
   Split into 1000-line chunks
2. `$ split -b 100M bigfile.tar.gz part_`
   Split into 100MB pieces
3. `$ split -n 4 data.csv segment_`
   Split into 4 equal parts
4. `$ split -d -l 500 log.txt log_`
   Numeric suffix chunks of 500 lines
5. `$ cat part_* > rejoined.txt`
   Rejoin split files

\$paste

paste [OPTION]... [FILE]...

Purpose

The paste command merges lines from multiple files side by side, joining corresponding lines with a tab delimiter by default. It is the horizontal complement to cat and is useful for combining columns from different data sources.

Syntax

```
paste [OPTION]... [FILE]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-d DELIM`	Use DELIM instead of tab as delimiter
`-s`	Paste one file at a time in serial (transpose lines)
`-z`	Use NUL as line delimiter instead of newline

Examples

1. `$ paste file1.txt file2.txt`
   Merge files side by side with tab
2. `$ paste -d',' col1.txt col2.txt`
   Merge with comma delimiter
3. `$ paste -s numbers.txt`
   Combine all lines of one file horizontally
4. `$ paste -d':' users.txt passwords.txt`
   Create user:pass pairs
5. `$ paste - - - < data.txt`
   Read stdin into 3 columns

\$cut

cut [OPTION]... [FILE]...

Purpose

The cut command extracts specific columns or fields from each line of text files. It is commonly used for processing CSV, TSV, and other delimited data, making it easy to extract specific columns from structured output.

Syntax

```
cut [OPTION]... [FILE]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-f LIST`	Select only these fields (comma-separated or range like 1-3)
`-d DELIM`	Use DELIM as field delimiter (default: tab)
`-c LIST`	Select characters by position
`-b LIST`	Select bytes by position
`--complement`	Invert selection: keep non-selected fields
`-s`	Do not print lines without delimiters

Examples

1. `$ cut -f1,3 data.tsv`
   Extract fields 1 and 3 (tab-delimited)
2. `$ cut -d',' -f2 file.csv`
   Extract second column from CSV
3. `$ cut -c1-10 file.txt`
   Extract first 10 characters of each line
4. `$ cut -d':' -f1 /etc/passwd`
   Get usernames from passwd file
5. `$ cut -d',' -f2- data.csv`
   Extract all columns from 2nd onwards

\$basename

basename NAME [SUFFIX]

Purpose

The basename command strips the directory path and optionally the file extension from a full pathname, returning just the filename component. It is frequently used in shell scripts to extract filenames for renaming, processing, or output labeling.

Syntax

```
basename NAME [SUFFIX]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
SUFFIX	If specified, strip this suffix from the result
-a	Support multiple arguments; treat each as a NAME
-s SUFFIX	Remove a trailing SUFFIX from each NAME
-z	Separate output items with NUL instead of newline

Examples

1. `$ basename /home/user/file.txt`
   Output: file.txt
2. `$ basename /etc/nginx/nginx.conf .conf`
   Output: nginx (strip .conf)
3. `$ basename -a /path/a.py /path/b.py`
   Output filenames for multiple paths
4. `$ basename -s .sh /usr/local/bin/deploy.sh`
   Strip .sh suffix
5. `$ for f in /src/*.py; do echo $(basename $f .py); done`
   Process Python filenames in loop

\$uuencode

uuencode [OPTION]... [INFILE] REMOTEFILE

Purpose

The uuencode command encodes binary files into ASCII text using the UU encoding scheme, making them safe for transmission over text-only channels such as email. The encoded output can be decoded back to binary using uudecode.

Syntax

```
uuencode [OPTION]... [INFILE] REMOTEFILE
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-m`	Use base64 encoding (MIME) instead of UU encoding
`-o FILE`	Write output to FILE instead of stdout

Examples

1. `$ uuencode binary.bin binary.bin > encoded.txt`
   Encode binary file
2. `$ uuencode -m image.jpg image.jpg > mail_attach.txt`
   Encode using base64
3. `$ uuencode file.tar.gz file.tar.gz | mail user@host`
   Email encoded attachment
4. `$ uuencode -m data.bin data.bin`
   Encode to stdout with base64
5. `$ uuencode logo.png logo.png -o logo_encoded.txt`
   Encode to output file

\$uudecode

uudecode [OPTION]... [FILE]...

Purpose

The uudecode command decodes files encoded by uuencode, restoring the original binary data. It reads the encoded ASCII text and reconstructs the binary file using the filename specified in the encoded file's header line.

Syntax

```
uudecode [OPTION]... [FILE]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-o FILE`	Write decoded output to FILE instead of encoded name
`-p`	Write output to stdout instead of a file
`-i`	Ignore lines not part of the encoding

Examples

1. `$ uudecode encoded.txt`
   Decode file to name stored in header
2. `$ uudecode -o output.bin encoded.txt`
   Decode to specified output filename
3. `$ uudecode -p encoded.txt > output.bin`
   Decode to stdout and redirect
4. `$ cat encoded.txt | uudecode -p > restored.bin`
   Pipeline decode
5. `$ uudecode attachment.txt`
   Decode email attachment

\$base32

base32 [OPTION]... [FILE]

Purpose

The base32 command encodes binary data using the Base32 encoding scheme, which represents binary data using 32 printable ASCII characters. It is commonly used in authentication tokens, TOTP codes, and data that must be readable and case-insensitive.

Syntax

```
base32 [OPTION]... [FILE]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-d`	Decode Base32-encoded data
`-i`	When decoding, ignore non-alphabet characters
`-w N`	Wrap encoded lines after N characters (default 76)
`-w 0`	Disable line wrapping

Examples

1. `$ base32 file.bin`
   Encode file to Base32
2. `$ base32 -d encoded.txt`
   Decode Base32 text to binary
3. `$ echo 'Hello' | base32`
   Encode string to Base32
4. `$ echo 'JBSWY3DP' | base32 -d`
   Decode Base32 back to text
5. `$ base32 -w 0 file.bin`
   Encode without line wrapping

\$base64

base64 [OPTION]... [FILE]

Purpose

The base64 command encodes binary data to Base64 ASCII representation or decodes Base64 text back to binary. It is widely used to encode binary data for embedding in text formats, HTTP headers, JSON, email, and shell scripts.

Syntax

```
base64 [OPTION]... [FILE]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-d`	Decode Base64-encoded data
`-i`	Ignore non-alphabet characters during decode
`-w N`	Wrap lines at N characters (default 76; 0=no wrap)
`-u`	Use URL-safe alphabet (GNU extension)

Examples

1. `$ base64 image.png > image.b64`
   Encode image to Base64
2. `$ base64 -d encoded.txt > original.bin`
   Decode Base64 back to binary
3. `$ echo 'Hello World' | base64`
   Encode a string
4. `$ echo 'SGVsbG8gV29ybGQ=' | base64 -d`
   Decode Base64 string
5. `$ base64 -w 0 cert.der > cert.b64`
   Encode without line wrap for embedding

\$cfdisk

cfdisk [OPTION]... DEVICE

Purpose

The cfdisk command provides a curses-based interactive partition table editor for disk drives. It offers a menu-driven interface for creating, deleting, resizing, and setting partition types on MBR and GPT disk layouts.

Syntax

```
cfdisk [OPTION]... DEVICE
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-z`	Start with zeroed partition table (create from scratch)
`-L`	Enable/disable output coloring

Examples

1. `$ cfdisk /dev/sda`
   Open partition editor for /dev/sda
2. `$ cfdisk /dev/sdb`
   Edit partitions on second disk
3. `$ cfdisk /dev/nvme0n1`
   Edit NVMe SSD partitions
4. `$ sudo cfdisk /dev/sda`
   Run with root privileges for writing
5. `$ cfdisk -z /dev/sdb`
   Start with an empty partition table

\$chattr

chattr [OPTION]... [MODE] FILE...

Purpose

The chattr command changes file attributes on ext2/ext3/ext4 filesystems beyond standard permissions. Attributes such as immutable (+i) prevent any modification to a file, even by root, until the attribute is removed. It is useful for hardening critical system files.

Syntax

```
chattr [OPTION]... [MODE] FILE...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`+i`	Set immutable: file cannot be modified, deleted, or renamed
`-i`	Remove immutable attribute
`+a`	Append only: file can only be appended to, not overwritten
`-a`	Remove append-only attribute
`+u`	Undeletable: file content saved when deleted
`-R`	Recursively change attributes of directory

Examples

1. `$ chattr +i /etc/resolv.conf`
   Make resolv.conf immutable
2. `$ chattr -i /etc/resolv.conf`
   Remove immutable flag to allow editing
3. `$ chattr +a /var/log/app.log`
   Set append-only on log file
4. `$ chattr -R +i /etc/ssh/`
   Protect all SSH config files
5. `$ lsattr /etc/resolv.conf`
   View attributes of the file

\$gawk

gawk [OPTION]... 'PROGRAM' [FILE]...

Purpose

The gawk command is the GNU implementation of awk, offering extensions over traditional awk including network access, interval expressions, advanced I/O, and profiling. It is the standard awk implementation on most Linux systems.

Syntax

```
gawk [OPTION]... 'PROGRAM' [FILE]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-F SEP`	Set input field separator
`-v VAR=VAL`	Set a variable value before execution
`-f FILE`	Read program from a file
`--profile`	Output profiling info to stdout
`BEGINFILE`	Pattern: execute before each input file
`ENDFILE`	Pattern: execute after each input file

Examples

1. `$ gawk '{print NR": "$0}' file.txt`
   Number all lines
2. `$ gawk -F: '{print $1,$3}' /etc/passwd`
   Print username and UID from passwd
3. `$ gawk 'length($0)>80' file.txt`
   Print lines longer than 80 characters
4. `$ gawk '{sum+=$1} END{print "Total:",sum}' data.txt`
   Sum column
5. `$ gawk 'BEGIN{FS=","} {print $2}' data.csv`
   CSV field extraction

\$getfacl

getfacl [OPTION]... FILE...

Purpose

The getfacl command retrieves and displays the Access Control Lists (ACLs) of files and directories. ACLs extend standard Unix permissions to allow fine-grained access control for specific users and groups beyond the owner/group/other model.

Syntax

```
getfacl [OPTION]... FILE...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-a`	Display ACL of files only (no default ACL)
`-d`	Display default ACL only (for directories)
`-c`	Omit the comment header from output
`-R`	Recursively list ACLs of directories
`--tabular`	Display output in tabular format
`-p`	Do not strip leading / from pathnames

Examples

1. `$ getfacl file.txt`
   Display ACL of file.txt
2. `$ getfacl -R /var/www`
   Recursively display all ACLs under /var/www
3. `$ getfacl -d /shared`
   Show default ACL of /shared directory
4. `$ getfacl /etc/app.conf | setfacl --restore=-`
   Backup and restore ACLs
5. `$ getfacl -c file.txt`
   Display ACL without header comment

\$ link

link FILE1 FILE2

Purpose

The link command creates a hard link to a file. Unlike ln, it is a minimal implementation that accepts exactly two arguments and calls the link() system call directly. It cannot create symbolic links or handle directories.

Syntax

```
link FILE1 FILE2
```

Examples

1. `$ link original.txt hardlink.txt`
   Create hard link to original.txt
2. `$ link /var/log/app.log /backup/app.log`
   Hard link a log file to backup
3. `$ link src.c backup_src.c`
   Create hard link of a source file
4. `$ link data.db data_snapshot.db`
   Hard link a database file
5. `$ ls -li original.txt hardlink.txt`
   Verify same inode number for both

\$cpio

cpio [OPTION]... [< or > ARCHIVE]

Purpose

The cpio command copies files to and from archives. It is used to create, extract, and list archive files, often in conjunction with find for flexible archiving. It is the format used by RPM packages and Linux initramfs images.

Syntax

```
cpio [OPTION]... [< or > ARCHIVE]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-o / --create`	Create an archive from stdin list
`-i / --extract`	Extract files from archive
`-p / --pass-through`	Copy files to another directory tree
`-v`	Verbose: list files being processed
`-d`	Create directories as needed during extraction
`-t`	Print table of contents (list archive)
`--format=FMT`	Use archive format: newc, odc, tar, crc

Examples

1. `$ find . | cpio -ov > archive.cpio`
   Create archive of current directory
2. `$ cpio -idv < archive.cpio`
   Extract archive in current directory
3. `$ cpio -t < archive.cpio`
   List contents without extracting
4. `$ find /src -name '*.conf' | cpio -pd /dst`
   Copy config files to /dst
5. `$ cpio -ov --format=newc < filelist > initramfs.cpio`
   Create initramfs-style archive

\$ locate

locate [OPTION]... PATTERN...

Purpose

The locate command searches a pre-built database index of all files on the system to quickly find files by name. It is much faster than find for simple name searches but requires the database to be periodically updated with updatedb.

Syntax

```
locate [OPTION]... PATTERN...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-i`	Case-insensitive search
`-c`	Print count of matching entries only
`-l N`	Limit output to N results
`-r REGEXP`	Search using a regular expression
`-n N`	Output at most N results
`--existing`	Only print results for files that currently exist
`-b`	Only match against the final component of pathnames

Examples

1. `$ locate nginx.conf`
   Find all files named nginx.conf
2. `$ locate -i '*.JPG'`
   Case-insensitive search for JPEG files
3. `$ locate -c '*.py'`
   Count Python files in database
4. `$ locate -r '/etc/.*\.conf$'`
   Use regex to find .conf files in /etc
5. `$ locate -b '\passwd'`
   Match exactly 'passwd' as basename

\$csplit

csplit [OPTION]... FILE PATTERN...

Purpose

The csplit command splits a file into sections determined by context lines matching patterns or line numbers. Unlike split which divides by size, csplit divides intelligently based on file content, making it useful for splitting log files or multi-record documents.

Syntax

```
csplit [OPTION]... FILE PATTERN...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-f PREFIX`	Use PREFIX instead of 'xx' for output filenames
`-n N`	Use N digits for output filename numbering
`-s`	Silent: do not print file sizes
`-k`	Keep output files even if an error occurs
`-b SUFFIX`	Use SUFFIX as output filename suffix

Examples

1. `$ csplit log.txt '/ERROR/' '{*}'`
   Split log on every ERROR line
2. `$ csplit -f chapter_ book.txt '/^Chapter/' '{*}'`
   Split book into chapters
3. `$ csplit data.txt 100 200`
   Split file at lines 100 and 200
4. `$ csplit -n 3 file.txt '/---/' '{5}'`
   Split 5 times with 3-digit numbering
5. `$ csplit -f part_ -b '%03d.txt' file '/---/' '{*}'`
   Named parts with formatted suffix

\$mktemp

mktemp [OPTION]... [TEMPLATE]

Purpose

The mktemp command creates temporary files or directories with a unique name, preventing name collision in concurrent scripts. It is essential for secure temporary file creation in shell scripts, avoiding predictable filenames that could be exploited.

Syntax

```
mktemp [OPTION]... [TEMPLATE]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-d`	Create a temporary directory
`-u`	Dry-run: print name but do not create
`-p DIR`	Create temp file in DIR
`-q`	Suppress error messages
`TEMPLATE`	Pattern with at least 3 X's at end, e.g., tmp.XXXXXX

Examples

1. `$ mktemp`
   Create temp file like /tmp/tmp.AbCdEf
2. `$ mktemp -d`
   Create a temporary directory
3. `$ mktemp /tmp/app.XXXXXX`
   Create temp file matching template
4. `$ TMPFILE=$(mktemp) && echo data > $TMPFILE`
   Use temp file in a script
5. `$ TMPDIR=$(mktemp -d) && chmod 700 $TMPDIR`
   Create secure temp directory

\$cd

cd [DIRECTORY]

Purpose

The cd command changes the current working directory. It is the primary way to navigate the filesystem in a shell session. Special shortcuts include ~ for home directory, - for previous directory, and .. for parent directory.

Syntax

```
cd [DIRECTORY]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`~`	Change to home directory
`-`	Change to previous working directory
`..`	Move up one directory level
`/`	Change to root directory
`PATH`	Change to specified absolute or relative path

Examples

1. `$ cd /var/log`
   Change to /var/log directory
2. `$ cd ~`
   Return to home directory
3. `$ cd ..`
   Move up one directory
4. `$ cd -`
   Return to previous directory
5. `$ cd /; ls`
   Change to root and list contents

\$netstat

netstat [OPTION]...

Purpose

The netstat command displays network connections, routing tables, interface statistics, masquerade connections, and multicast memberships. It is used to diagnose network problems and verify active connections on a system. On modern systems, ss is preferred.

Syntax

```
netstat [OPTION]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-t`	Display TCP connections only
`-u`	Display UDP connections only
`-l`	Show only listening sockets
`-n`	Numeric addresses instead of resolving names
`-p`	Show PID and program name for each socket
`-r`	Display routing table
`-s`	Display per-protocol statistics
`-a`	Show all connections and listening ports

Examples

1. `$ netstat -tuln`
   List all listening TCP/UDP ports numerically
2. `$ netstat -anp | grep ':80'`
   Find process using port 80
3. `$ netstat -rn`
   Display routing table
4. `$ netstat -s`
   Show network statistics per protocol
5. `$ netstat -p | grep python`
   Find all connections made by Python

\$rsh

rsh [OPTION]... HOST [COMMAND]

Purpose

The rsh (remote shell) command executes a command on a remote host. It is a legacy insecure remote access tool that has been largely replaced by SSH. rsh transmits data and credentials in plain text and should not be used on untrusted networks.

Syntax

```
rsh [OPTION]... HOST [COMMAND]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-l USER`	Login as USER on remote host
`-n`	Redirect stdin from /dev/null
`-d`	Debug socket operations

Examples

1. `$ rsh remotehost ls -l`
   Run ls on remote host
2. `$ rsh -l admin server01 df -h`
   Run df as admin on server01
3. `$ rsh host 'cat /etc/hosts'`
   Read remote hosts file
4. `$ rsh server 'ps aux'`
   List processes on remote server
5. `$ rsh -l root host 'service nginx restart'`
   Restart nginx remotely

\$ssh

ssh [OPTION]... [USER@]HOST [COMMAND]

Purpose

The ssh command provides secure encrypted remote login and command execution over a network. It uses public-key cryptography to authenticate and encrypts all traffic, making it the standard tool for remote system administration and secure file transfers.

Syntax

```
ssh [OPTION]... [USER@]HOST [COMMAND]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-i FILE`	Identity file (private key) for authentication
`-p PORT`	Connect to PORT instead of default 22
`-L PORT:HOST:PORT`	Local port forwarding
`-R PORT:HOST:PORT`	Remote port forwarding
`-D PORT`	Dynamic SOCKS proxy
`-N`	Do not execute remote command (port forwarding only)
`-X`	Enable X11 forwarding
`-v`	Verbose debug output

Examples

1. `$ ssh user@192.168.1.10`
   Connect to remote host
2. `$ ssh -p 2222 admin@server.com`
   Connect on non-standard port
3. `$ ssh -i ~/.ssh/id_rsa user@host`
   Use specific private key
4. `$ ssh -L 8080:localhost:80 user@host`
   Forward local port 8080 to remote port 80
5. `$ ssh user@host 'df -h'`
   Run single command remotely

\$nmap

nmap [OPTION]... TARGET

Purpose

The nmap command is a powerful network scanner used for discovering hosts, services, open ports, operating systems, and security vulnerabilities. It is the standard tool for network auditing, security assessments, and understanding network topology.

Syntax

```
nmap [OPTION]... TARGET
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-sS`	TCP SYN (stealth) scan
`-sV`	Probe open ports to determine service/version
`-O`	Enable OS detection
`-p PORTS`	Specify port range (e.g., 1-1000 or 80,443)
`-A`	Aggressive: OS detection, version, scripts, traceroute
`-Pn`	Treat hosts as online; skip host discovery
`-T4`	Faster timing template
`--script NAME`	Run Nmap Scripting Engine script

Examples

1. `$ nmap 192.168.1.1`
   Basic scan of a single host
2. `$ nmap -sV -p 1-1000 192.168.1.0/24`
   Scan subnet ports 1-1000 with version
3. `$ nmap -A -T4 scanme.nmap.org`
   Aggressive scan with OS detection
4. `$ nmap -Pn --script vuln target`
   Scan for vulnerabilities
5. `$ nmap -p 22,80,443 192.168.1.100`
   Scan specific ports only

\$ifconfig

ifconfig [INTERFACE] [OPTION]...

Purpose

The ifconfig command configures and displays network interface parameters including IP address, netmask, broadcast address, and MAC address. It is the traditional Unix network configuration tool, now largely replaced by the ip command on modern Linux systems.

Syntax

```
ifconfig [INTERFACE] [OPTION]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`up`	Activate a network interface
`down`	Deactivate a network interface
`add ADDR`	Add IP address to interface
`netmask MASK`	Set subnet mask
`broadcast ADDR`	Set broadcast address
`mtu N`	Set Maximum Transmission Unit size

Examples

1. **`$ ifconfig`**
   Display all active network interfaces

2. **`$ ifconfig eth0`**
   Show details for eth0 interface

3. **`$ ifconfig eth0 192.168.1.100`**
   Assign IP address to eth0

4. **`$ ifconfig eth0 down`**
   Bring down network interface

5. **`$ ifconfig eth0 mtu 1500`**
   Set MTU for eth0

\$ping

ping [OPTION]... DESTINATION

Purpose

The ping command tests network connectivity to a host by sending ICMP echo request packets and measuring the round-trip time. It is the first tool used to diagnose network connectivity issues and measure network latency between hosts.

Syntax

```
ping [OPTION]... DESTINATION
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-c N`	Stop after sending N packets
`-i SEC`	Wait SEC seconds between packets
`-s SIZE`	Set packet size in bytes
`-t TTL`	Set IP Time-To-Live
`-W TIMEOUT`	Time to wait for a response (seconds)
`-q`	Quiet: only show summary
`-f`	Flood ping: send as fast as possible (root only)

Examples

1. `$ ping google.com`
   Ping Google continuously
2. `$ ping -c 4 192.168.1.1`
   Send exactly 4 ping packets
3. `$ ping -c 3 -i 2 host`
   3 pings with 2-second intervals
4. `$ ping -s 1472 gateway`
   Ping with large packet (test MTU)
5. `$ ping -q -c 100 server.local`
   Quiet 100-packet latency test

\$ip

ip [OPTION]... OBJECT COMMAND

Purpose

The ip command is the modern replacement for ifconfig and route, providing comprehensive control over network interfaces, routing tables, ARP tables, tunnels, and more. It is part of the iproute2 package and is the standard network tool on Linux.

Syntax

```
ip [OPTION]... OBJECT COMMAND
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`addr`	Manage IP addresses on interfaces
`link`	Manage network interfaces (bring up/down, MTU)
`route`	Manage routing tables
`neigh`	Manage ARP/neighbour tables
`-4`	Show only IPv4 information
`-6`	Show only IPv6 information
`-s`	Show statistics
`-br`	Brief output format

Examples

1. `$ ip addr show`
   Show all IP addresses
2. `$ ip link set eth0 up`
   Bring up network interface
3. `$ ip addr add 192.168.1.10/24 dev eth0`
   Add IP address to interface
4. `$ ip route show`
   Display routing table
5. `$ ip route add default via 192.168.1.1`
   Add default gateway

\$iptables

iptables [OPTION]... RULE-SPECIFICATION

Purpose

The iptables command manages the Linux kernel firewall rules for IPv4 traffic. It filters, NATs, and mangles network packets based on defined rules, organized in chains (INPUT, OUTPUT, FORWARD) and tables (filter, nat, mangle).

Syntax

```
iptables [OPTION]... RULE-SPECIFICATION
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-A CHAIN`	Append rule to chain
`-I CHAIN N`	Insert rule at position N in chain
`-D CHAIN`	Delete rule from chain
`-L`	List all rules
`-F`	Flush (delete) all rules in chain
`-P CHAIN TARGET`	Set default policy for chain
`-s IP`	Match source IP address
`-p PROTO`	Match protocol (tcp, udp, icmp)

Examples

1. `$ iptables -L -n -v`
   List all firewall rules verbosely
2. `$ iptables -A INPUT -p tcp --dport 22 -j ACCEPT`
   Allow SSH traffic
3. `$ iptables -A INPUT -p tcp --dport 80 -j ACCEPT`
   Allow HTTP traffic
4. `$ iptables -P INPUT DROP`
   Set default INPUT policy to DROP
5. `$ iptables -A INPUT -m state --state ESTABLISHED -j ACCEPT`
   Allow established connections

\$dig

dig [OPTIONS] [@SERVER] NAME [TYPE]

Purpose

The dig command (Domain Information Groper) queries DNS servers to resolve domain names and retrieve various DNS record types. It provides detailed DNS query/response information, making it the preferred tool for DNS troubleshooting and verification.

Syntax

```
dig [OPTIONS] [@SERVER] NAME [TYPE]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`@SERVER`	DNS server to query (e.g., @8.8.8.8)
`A`	Query A (IPv4 address) record
`MX`	Query mail exchanger records
`NS`	Query name server records
`CNAME`	Query canonical name record
`TXT`	Query text records
`-x IP`	Reverse DNS lookup
`+short`	Display only the answer, no headers

Examples

1. `$ dig google.com`
   Query A record for google.com
2. `$ dig @8.8.8.8 example.com`
   Query using Google's DNS server
3. `$ dig example.com MX`
   Get mail server records
4. `$ dig -x 8.8.8.8`
   Reverse DNS lookup for IP
5. `$ dig +short example.com`
   Short output — just the IP

\$clear

clear

Purpose

The clear command clears the terminal screen, removing all previous output from view. It sends the appropriate escape sequence to the terminal to refresh the display, providing a clean working environment without closing the terminal.

Syntax

```
clear
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-x`	Do not clear scrollback (only visible area)

Examples

1. `$ clear`
   Clear the terminal screen
2. `$ clear; ls`
   Clear screen then list files
3. `$ alias cls='clear'`
   Create alias cls for clear
4. `$ ctrl+l`
   Keyboard shortcut equivalent to clear
5. `$ clear && date`
   Clear and show current date

\$env

env [OPTION]... [NAME=VALUE]... [COMMAND]

Purpose

The env command runs a program in a modified environment or displays the current environment variables. It can add, remove, or override environment variables for a single command without permanently changing the shell environment.

Syntax

```
env [OPTION]... [NAME=VALUE]... [COMMAND]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-i`	Start with an empty environment
`-u NAME`	Remove variable NAME from environment
`NAME=VALUE`	Set environment variable for command
`--`	End of options; treat remaining as command

Examples

1. **`$ env`**
   Print all current environment variables

2. **`$ env PATH=/custom/bin app`**
   Run app with custom PATH

3. **`$ env -i PATH=/bin:/usr/bin bash`**
   Start bash with minimal environment

4. **`$ env -u HOME python script.py`**
   Run script without HOME set

5. **`$ env LANG=C grep pattern file`**
   Run grep with C locale

\$exit

exit [N]

Purpose

The exit command exits the current shell session or terminates a script. An optional numeric exit code (0-255) can be provided, where 0 indicates success and any non-zero value indicates an error, which can be checked by parent processes.

Syntax

```
exit [N]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
0	Success exit code
1	General error exit code
2	Misuse of shell command
N	Custom exit code (0-255)

Examples

1. `$ exit`
   Exit with last command's status
2. `$ exit 0`
   Exit with success code
3. `$ exit 1`
   Exit with error code 1
4. `$ if ! cmd; then exit 1; fi`
   Exit script on command failure
5. `$ trap 'exit 0' INT; while true; do sleep 1; done`
   Exit cleanly on Ctrl+C

\$ login

login [OPTION]... [USERNAME]

Purpose

The login command authenticates a user and starts a new login shell session. It prompts for a username and password, validates credentials against /etc/passwd and /etc/shadow, sets up the user's environment, and starts their configured shell.

Syntax

```
login [OPTION]... [USERNAME]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-p`	Preserve environment instead of using the user's
`-f`	Skip authentication (already verified)
`-h HOST`	Name of remote host for authentication logging

Examples

1. **`$ login`**
   Start a new login prompt

2. **`$ login alice`**
   Log in as user alice

3. **`$ su - alice`**
   Switch user and simulate login

4. **`$ login -f alice`**
   Force login without password check

5. **`$ ssh user@host 'login'`**
   Trigger login on remote machine

\$history

history [OPTION]... [N]

Purpose

The history command displays or manipulates the command history list of the current shell session. It allows reviewing past commands, re-executing them by number, searching the history, and configuring history behavior through environment variables.

Syntax

```
history [OPTION]... [N]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
N	Show last N commands
-c	Clear all history entries
-d OFFSET	Delete history entry at position OFFSET
-a	Append session history to history file
-r	Read history file and append to session history
-w	Write current history to history file

Examples

1. **$ history**
   Show full command history
2. **$ history 20**
   Show last 20 commands
3. **$!45**
   Re-execute command number 45
4. **$!!**
   Re-execute last command
5. **$ history -c**
   Clear command history

\$sleep

sleep NUMBER[SUFFIX]...

Purpose

The sleep command pauses execution for a specified duration. It is widely used in shell scripts to add delays between operations, implement retry logic, create timing loops, and schedule tasks within scripts.

Syntax

```
sleep NUMBER[SUFFIX]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
s	Seconds (default suffix)
m	Minutes
h	Hours
d	Days

Examples

1. `$ sleep 5`
   Pause for 5 seconds
2. `$ sleep 0.5`
   Pause for 500 milliseconds
3. `$ sleep 1m`
   Pause for 1 minute
4. `$ sleep 2h`
   Pause for 2 hours
5. `$ while true; do check_service; sleep 30; done`
   Run check every 30 seconds

\$ su

su [OPTION]... [USER]

Purpose

The su command allows switching to another user account within the current shell session. When used with the dash (-) option, it simulates a full login for the target user, loading their environment, shell profile, and working directory.

Syntax

```
su [OPTION]... [USER]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-`	Simulate a full login for the target user
`-c CMD`	Execute CMD as the specified user
`-s SHELL`	Use SHELL instead of user's default shell
`-l`	Same as - option: simulate login
`-p`	Preserve current environment

Examples

1. `$ su root`
   Switch to root user
2. `$ su -`
   Full login switch to root
3. `$ su - alice`
   Full login switch to user alice
4. `$ su -c 'apt update' root`
   Run command as root
5. `$ su - postgres -c 'psql'`
   Run psql as postgres user

\$sudo

sudo [OPTION]... COMMAND

Purpose

The sudo command runs commands with the security privileges of another user, typically root. It provides controlled access to administrator functions by authenticating the current user's password and checking sudoers policy, logging all activity.

Syntax

```
sudo [OPTION]... COMMAND
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-u USER`	Run command as USER instead of root
`-i`	Run shell as root with login environment
`-s`	Run shell as target user
`-l`	List allowed sudo commands
`-k`	Invalidate cached credentials
`-E`	Preserve current user's environment

Examples

1. `$ sudo apt update`
   Run apt update as root
2. `$ sudo -u postgres psql`
   Run psql as postgres user
3. `$ sudo -i`
   Start interactive root shell
4. `$ sudo !!`
   Re-run last command as root
5. `$ sudo -l`
   List your sudo privileges

\$nc

nc [OPTION]... HOST PORT

Purpose

The nc (netcat) command is a versatile networking tool that reads and writes data across network connections using TCP or UDP. Often called the 'Swiss Army knife' of networking, it is used for port scanning, file transfer, debugging, and creating backdoors in testing.

Syntax

```
nc [OPTION]... HOST PORT
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-l`	Listen mode: accept incoming connection
`-u`	Use UDP instead of TCP
`-p PORT`	Specify source port
`-v`	Verbose: output connection info
`-z`	Zero-I/O mode: just scan for listening daemons
`-w SEC`	Connection timeout in seconds
`-e CMD`	Execute CMD after connection

Examples

1. `$ nc -zv host 80`
   Check if port 80 is open
2. `$ nc -l 4444`
   Listen on port 4444
3. `$ nc host 4444 < file.txt`
   Send file over network
4. `$ echo 'GET / HTTP/1.0' | nc web.com 80`
   Manual HTTP request
5. `$ nc -l 1234 | tar -xzf -`
   Receive and extract tar over network

\$nslookup

nslookup [OPTION]... [HOST] [SERVER]

Purpose

The nslookup command queries DNS servers for domain name and IP address mapping. It can perform forward lookups (domain to IP), reverse lookups (IP to domain), and query specific record types such as MX, NS, and CNAME.

Syntax

```
nslookup [OPTION]... [HOST] [SERVER]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-type=TYPE`	Query record type: A, MX, NS, CNAME, PTR, TXT
`-timeout=N`	Set query timeout in seconds
`-retry=N`	Set number of retries

Examples

1. `$ nslookup google.com`
   Forward DNS lookup for google.com
2. `$ nslookup 8.8.8.8`
   Reverse lookup for IP 8.8.8.8
3. `$ nslookup -type=MX gmail.com`
   Get mail server records for Gmail
4. `$ nslookup -type=NS example.com`
   Get nameservers for example.com
5. `$ nslookup google.com 1.1.1.1`
   Query using Cloudflare's DNS

\$wget

wget [OPTION]... URL...

Purpose

The wget command downloads files from the web using HTTP, HTTPS, and FTP protocols. It supports recursive downloading, bandwidth throttling, resuming interrupted downloads, and operating without user interaction, making it ideal for scripts and automation.

Syntax

```
wget [OPTION]... URL...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-O FILE`	Save downloaded content to FILE
`-q`	Quiet mode: suppress output
`-r`	Recursive download
`-c`	Continue (resume) a partial download
`-P DIR`	Save files to PREFIX directory
`--limit-rate=RATE`	Limit download speed
`-np`	No parent directories (recursive)
`-b`	Background download

Examples

1. `$ wget https://example.com/file.zip`
   Download a file
2. `$ wget -O output.zip https://url/file`
   Save with custom filename
3. `$ wget -c https://large-file.iso`
   Resume interrupted download
4. `$ wget -r -np https://site.com/docs/`
   Recursively download docs directory
5. `$ wget -q --spider https://site.com`
   Check if URL exists without downloading

\$traceroute

traceroute [OPTION]... HOST

Purpose

The traceroute command shows the network path (route) that packets take from your host to a destination, displaying each intermediate router (hop) along the way with its IP address and round-trip time. It is used to diagnose routing issues and network latency.

Syntax

```
traceroute [OPTION]... HOST
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-n`	Numeric: do not resolve hostnames
`-m N`	Set maximum number of hops
`-w N`	Wait N seconds for each response
`-q N`	Number of probes per hop (default 3)
`-p PORT`	Use specified destination port
`-I`	Use ICMP instead of UDP

Examples

1. `$ traceroute google.com`
   Trace route to Google
2. `$ traceroute -n 8.8.8.8`
   Trace with numeric output only
3. `$ traceroute -m 20 example.com`
   Limit to 20 hops
4. `$ traceroute -I example.com`
   Use ICMP packets (like Windows tracert)
5. `$ traceroute -q 1 host`
   Faster trace with 1 probe per hop

\$ss

ss [OPTION]... [FILTER]

Purpose

The ss command (socket statistics) is the modern replacement for netstat, displaying detailed information about network sockets. It is faster and provides more information, including filtering by state, protocol, port, and process ID.

Syntax

```
ss [OPTION]... [FILTER]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-t`	Show TCP sockets
`-u`	Show UDP sockets
`-l`	Show listening sockets
`-n`	Numeric addresses and port numbers
`-p`	Show process using the socket
`-a`	Show all sockets (listening and established)
`-s`	Show socket summary statistics
`-4 / -6`	Show IPv4 / IPv6 sockets only

Examples

1. `$ ss -tuln`
   All listening TCP/UDP sockets numerically
2. `$ ss -tp`
   TCP sockets with owning processes
3. `$ ss -s`
   Show socket statistics summary
4. `$ ss -tnp | grep ':443'`
   Find process on HTTPS port
5. `$ ss -u -a`
   All UDP sockets including non-listening

\$mtr

mtr [OPTION]... HOST

Purpose

The mtr command combines the functionality of ping and traceroute in a real-time continuously updating display. It shows all network hops to a destination along with per-hop packet loss and latency statistics, making it ideal for diagnosing intermittent network issues.

Syntax

```
mtr [OPTION]... HOST
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-n`	Numeric: do not resolve hostnames
`-c N`	Send N pings and quit
`-r`	Report mode: print stats and exit
`-s SIZE`	Packet size in bytes
`-i INTERVAL`	Ping interval in seconds
`-b`	Show both IP and hostname

Examples

1. `$ mtr google.com`
   Live network path to Google
2. `$ mtr -n 8.8.8.8`
   Numeric live traceroute
3. `$ mtr -r -c 20 host`
   Report mode with 20 cycles
4. `$ mtr --report-wide host`
   Wide output report
5. `$ mtr -n --report -c 50 host > mtr.txt`
   Save report to file

\$ifdown

ifdown [OPTION]... INTERFACE

Purpose

The ifdown command brings down a network interface by calling the scripts specified for the interface in /etc/network/interfaces. It gracefully terminates the network connection on the specified interface, releasing IP addresses and stopping services.

Syntax

```
ifdown [OPTION]... INTERFACE
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-a`	Bring down all interfaces marked 'auto'
`-v`	Verbose: show commands as run
`-n`	Dry-run: show what would happen
`--force`	Force bringing down even if config missing

Examples

1. `$ ifdown eth0`
   Bring down eth0 interface
2. `$ ifdown -a`
   Bring down all auto interfaces
3. `$ ifdown -v wlan0`
   Bring down WiFi verbosely
4. `$ ifdown --force eth1`
   Force eth1 down
5. `$ ifdown eth0 && ifup eth0`
   Restart network interface

\$ifup

ifup [OPTION]... INTERFACE

Purpose

The ifup command activates a network interface according to the configuration in /etc/network/interfaces. It sets the IP address, gateway, and runs any pre-up/post-up scripts defined for the interface.

Syntax

```
ifup [OPTION]... INTERFACE
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-a`	Bring up all interfaces marked 'auto'
`-v`	Verbose: show commands being run
`-n`	Dry-run: do not actually configure
`--force`	Force bring up even if interface is up

Examples

1. `$ ifup eth0`
   Bring up eth0 interface
2. `$ ifup -a`
   Bring up all auto interfaces
3. `$ ifup -v wlan0`
   Bring up WiFi with verbose output
4. `$ ifdown eth0 && ifup eth0`
   Restart interface
5. `$ ifup --force eth0`
   Force interface up

\$curl

curl [OPTION]... URL

Purpose

The curl command transfers data to or from servers using various protocols including HTTP, HTTPS, FTP, SFTP, and more. It is extremely versatile, supporting authentication, cookies, custom headers, file uploads, and is widely used in DevOps automation and API testing.

Syntax

```
curl [OPTION]... URL
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-o FILE`	Save output to FILE
`-O`	Save with remote filename
`-L`	Follow redirects
`-s`	Silent: suppress progress and errors
`-H 'HEADER'`	Add custom request header
`-d DATA`	POST data (form body)
`-X METHOD`	Specify HTTP method (GET, POST, PUT, DELETE)
`-u USER:PASS`	HTTP basic authentication

Examples

1. `$ curl https://api.example.com/data`
   GET request to API
2. `$ curl -O https://example.com/file.zip`
   Download file with original name
3. `$ curl -X POST -H 'Content-Type: application/json' -d '{"key":"val"}' https://api.com`
   POST JSON
4. `$ curl -u admin:pass https://private.example.com`
   Authenticated request
5. `$ curl -I https://example.com`
   Show only HTTP response headers

\$clock

clock [OPTION]...

Purpose

The clock command displays or sets the hardware (RTC) system clock. It reads the BIOS/hardware clock time and can synchronize it with the system clock. It is often used to correct time drift and ensure the hardware clock is accurate after changes.

Syntax

```
clock [OPTION]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`--show`	Display hardware clock time
`--set --date='STRING'`	Set hardware clock to given date/time
`--hctosys`	Set system time from hardware clock
`--systohc`	Set hardware clock from system time
`--utc`	Hardware clock uses UTC time
`--localtime`	Hardware clock uses local time

Examples

1. `$ clock --show`
   Display current hardware clock time
2. `$ clock --systohc`
   Sync hardware clock from system time
3. `$ clock --hctosys`
   Set system time from hardware clock
4. `$ clock --set --date='2026-01-01 12:00:00'`
   Set hardware clock manually
5. `$ hwclock --show`
   Modern equivalent using hwclock

\$free

free [OPTION]...

Purpose

The free command displays memory usage statistics including total, used, free, shared, buffer/cache, and available memory for both RAM and swap space. It is an essential tool for monitoring system memory health and diagnosing memory pressure.

Syntax

```
free [OPTION]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-h`	Human-readable: display in KB, MB, GB
`-m`	Display in megabytes
`-g`	Display in gigabytes
`-s N`	Continuously display every N seconds
`-c N`	Display N times then exit
`-t`	Show total line summing RAM and swap
`-w`	Wide output: separate buffers and cache

Examples

1. `$ free -h`
   Show memory in human-readable format
2. `$ free -m`
   Show memory in megabytes
3. `$ free -h -s 2`
   Update every 2 seconds
4. `$ free -t`
   Show totals including swap
5. `$ watch -n 1 free -h`
   Live memory monitoring

\$df

df [OPTION]... [FILE]...

Purpose

The df command reports the amount of disk space used and available on mounted filesystems. It shows total size, used space, available space, and percentage of use for each mounted volume. It is essential for capacity planning and monitoring disk usage.

Syntax

```
df [OPTION]... [FILE]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-h`	Human-readable sizes (KB, MB, GB)
`-H`	Human-readable using powers of 1000
`-T`	Show filesystem type
`-t TYPE`	Show only filesystems of TYPE
`-x TYPE`	Exclude filesystems of TYPE
`-i`	Show inode usage instead of block usage
`-a`	Include pseudo, duplicate, inaccessible filesystems

Examples

1. `$ df -h`
   Show disk space in human-readable format
2. `$ df -h /home`
   Show space for /home filesystem only
3. `$ df -T`
   Show disk space with filesystem type
4. `$ df -i`
   Show inode usage
5. `$ df -h --total`
   Show totals row at the bottom

\$du

du [OPTION]... [FILE]...

Purpose

The du command estimates file and directory space usage. It recursively calculates disk consumption for directories, helping identify large directories and files that are consuming disk space. It is commonly used for disk cleanup and capacity management.

Syntax

```
du [OPTION]... [FILE]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-h`	Human-readable sizes
`-s`	Summarize: display total for each argument only
`-a`	Show sizes for all files, not just directories
`--max-depth=N`	Limit directory depth to N levels
`-c`	Grand total at end of output
`-x`	Do not cross filesystem boundaries
`--exclude=PAT`	Exclude files matching PATTERN

Examples

1. `$ du -h /var/log`
   Show disk usage of /var/log
2. `$ du -sh *`
   Summarize size of each item in current dir
3. `$ du -h --max-depth=1 /`
   Top-level directory sizes from root
4. `$ du -sh /home/* | sort -h`
   Sort home directories by size
5. `$ du -sh --exclude='*.log' /var`
   Exclude log files from calculation

\$crontab

crontab [OPTION]... [FILE]

Purpose

The crontab command manages scheduled tasks (cron jobs) that run automatically at specified times. Each user can have their own crontab file defining when and which commands to execute automatically, from every minute to annually.

Syntax

```
crontab [OPTION]... [FILE]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-e`	Edit the current user's crontab
`-l`	List the current user's crontab
`-r`	Remove the current user's crontab
`-u USER`	Specify user whose crontab to manage
`-i`	Prompt before deleting crontab

Examples

1. `$ crontab -e`
   Edit your crontab in default editor
2. `$ crontab -l`
   List your scheduled cron jobs
3. `$ crontab -r`
   Remove all your cron jobs
4. `$ echo '0 2 * * * /backup.sh' | crontab -`
   Set cron job via pipe
5. `$ crontab -u alice -l`
   List cron jobs for user alice

\$finger

finger [OPTION]... [USER[@HOST]]

Purpose

The finger command displays information about system users, including login name, full name, terminal, login time, and other information from the GECOS field of /etc/passwd. It can also query remote fingerd servers for information about remote users.

Syntax

```
finger [OPTION]... [USER[@HOST]]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-l`	Long format with more details
`-s`	Short format (default)
`-p`	Do not display .plan and .project files
`-m`	Do not match against real names

Examples

1. `$ finger`
   Show all logged-in users
2. `$ finger alice`
   Show info about user alice
3. `$ finger alice@remote.host`
   Query remote user info
4. `$ finger -l bob`
   Detailed info about bob
5. `$ finger -s`
   Short format for all users

\$ last

last [OPTION]... [USER] [TTY]

Purpose

The last command shows a listing of last logged in users, reading from the /var/log/wtmp binary file. It displays user, terminal, remote host, login time, logout time, and session duration, helping administrators audit user access history.

Syntax

```
last [OPTION]... [USER] [TTY]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-n N`	Show only N lines of output
`-a`	Display hostname in last column
`-F`	Full date and time for all fields
`-x`	Show system shutdown/reboot entries
`-R`	Do not display hostname
`-f FILE`	Read from FILE instead of /var/log/wtmp

Examples

1. **`$ last`**
   Show all recent logins

2. **`$ last alice`**
   Show logins for user alice

3. **`$ last -n 20`**
   Show last 20 login records

4. **`$ last reboot`**
   Show all reboot records

5. **`$ last -F root`**
   Show root logins with full timestamps

\$ lpq

lpq [OPTION]... [JOB_ID...]

Purpose

The lpq command displays the contents of the print queue, showing pending, active, and completed print jobs. It allows users to check if their documents are in the queue and see the status of current print jobs. Multiple printers can be queried.

Syntax

```
lpq [OPTION]... [JOB_ID...]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-P PRINTER`	Specify printer queue to query
`-a`	Show jobs on all printers
`-l`	Long listing format with more details
`-U USER`	Show jobs for specific username

Examples

1. `$ lpq`
   Show default printer queue
2. `$ lpq -a`
   Show jobs on all printers
3. `$ lpq -P HP_LaserJet`
   Show queue for specific printer
4. `$ lpq -l`
   Long format job listing
5. `$ lpq -U alice`
   Show alice's print jobs

\$man

man [OPTION]... [SECTION] NAME

Purpose

The man command displays the system reference manual pages (man pages) for commands, system calls, library functions, and configuration files. Man pages are the primary documentation source on Unix-like systems, organized into numbered sections.

Syntax

```
man [OPTION]... [SECTION] NAME
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-k KEYWORD`	Search all man pages for keyword (apropos)
`-f CMD`	Display one-line description (whatis)
`-a`	Show all matching man pages
`-w`	Print locations of man pages
`SECTION`	1=commands, 2=syscalls, 3=library, 5=files, 8=admin

Examples

1. `$ man ls`
   Show manual page for ls
2. `$ man 5 passwd`
   Show section 5 (file format) for passwd
3. `$ man -k 'copy file'`
   Search for commands related to copying files
4. `$ man -f tar`
   Show one-line description of tar
5. `$ man 7 regex`
   Show regex reference documentation

\$path

PATH=$PATH:/new/directory

Purpose

The PATH environment variable specifies a list of directories where the shell searches for executable commands. Managing PATH is essential for making programs accessible from the command line, customizing which versions of tools are used, and integrating new software.

Syntax

```
PATH=$PATH:/new/directory
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`$PATH`	Existing path variable (append using :$PATH)
`export PATH`	Make PATH available to subprocesses

Examples

1. `$ echo $PATH`
   Display current PATH
2. `$ export PATH=$PATH:/opt/myapp/bin`
   Add directory to PATH
3. `$ export PATH=/usr/local/bin:$PATH`
   Prepend directory to PATH
4. `$ which python`
   Find which python is in PATH
5. `$ env | grep PATH`
   Show PATH from environment

\$printenv

printenv [OPTION]... [VARIABLE]...

Purpose

The printenv command prints the values of environment variables. If no variable names are given, it prints all environment variables and their values. It is useful in scripts to check variable states and debug environment configurations.

Syntax

```
printenv [OPTION]... [VARIABLE]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-0`	End output with NUL byte instead of newline

Examples

1. **`$ printenv`**
   Print all environment variables
2. **`$ printenv HOME`**
   Print value of HOME variable
3. **`$ printenv PATH`**
   Print current PATH value
4. **`$ printenv LANG TERM`**
   Print multiple variables
5. **`$ printenv | sort`**
   Print sorted environment variables

\$ps

ps [OPTION]...

Purpose

The ps command reports a snapshot of current running processes, showing process IDs, terminal, CPU and memory usage, and command names. It is fundamental for system monitoring, identifying runaway processes, and understanding the system process hierarchy.

Syntax

```
ps [OPTION]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-e / -A`	Select all processes
`-f`	Full-format listing with more columns
`-u USER`	Show processes for specific user
`-p PID`	Show specific process by PID
`aux`	Show all processes with user-oriented format
`-l`	Long format with more information
`--forest`	Show process tree with ASCII art
`-o FORMAT`	Specify custom output columns

Examples

1. `$ ps aux`
   Show all processes (BSD style)
2. `$ ps -ef`
   Show all processes (System V style)
3. `$ ps -u alice`
   Show processes for user alice
4. `$ ps aux | grep nginx`
   Find nginx processes
5. `$ ps --forest`
   Show process tree hierarchy

\$set

set [OPTION]... [ARGUMENT]...

Purpose

The set command displays or modifies shell options and positional parameters. It can enable or disable shell features (like exit on error with -e), display all variables and functions, and set positional parameters for shell scripts.

Syntax

```
set [OPTION]... [ARGUMENT]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-e`	Exit immediately if a command fails
`-x`	Print each command before executing (trace)
`-u`	Treat unset variables as errors
`-o pipefail`	Fail if any command in a pipeline fails
`-v`	Print input lines as they are read
`-n`	Read commands but do not execute
`-f`	Disable filename globbing

Examples

1. `$ set`
   Display all shell variables and functions
2. `$ set -e`
   Enable exit on error
3. `$ set -x`
   Enable command tracing for debugging
4. `$ set -euo pipefail`
   Strict mode for shell scripts
5. `$ set -- arg1 arg2`
   Set positional parameters

\$ionice

ionice [OPTION]... COMMAND | -p PID

Purpose

The ionice command sets or gets the I/O scheduling class and priority of a process. It controls how much disk I/O a process receives, allowing administrators to reduce I/O priority for background tasks so they do not impact interactive workloads.

Syntax

```
ionice [OPTION]... COMMAND | -p PID
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-c CLASS`	I/O class: 1=realtime, 2=best-effort, 3=idle
`-n N`	Priority within class (0=highest, 7=lowest)
`-p PID`	Set priority for existing process by PID
`-t`	Ignore failures

Examples

1. `$ ionice -c 3 find / -name '*.log'`
   Run find at idle I/O priority
2. `$ ionice -c 2 -n 7 rsync -a /src /dst`
   Low priority rsync backup
3. `$ ionice -p 1234`
   Get I/O class of process 1234
4. `$ ionice -c 1 -n 4 -p 1234`
   Set real-time I/O for process
5. `$ ionice -c 3 make -j4`
   Build at idle priority

\$time

time COMMAND [ARGUMENTS]

Purpose

The time command measures the execution time of a command, reporting real (wall-clock) time, user CPU time, and system CPU time. It is useful for benchmarking, performance tuning, and understanding where time is spent in complex operations.

Syntax

```
time COMMAND [ARGUMENTS]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-p`	POSIX format output
`-o FILE`	Append timing output to FILE
`-v`	Verbose: display extensive resource usage

Examples

1. `$ time ls -R /`
   Time recursive directory listing
2. `$ time python script.py`
   Measure Python script execution time
3. `$ time dd if=/dev/zero of=/tmp/test bs=1M count=100`
   Benchmark disk write speed
4. `$ time find / -name '*.conf'`
   Time a find operation
5. `$ /usr/bin/time -v command`
   Verbose time with memory stats

\$uptime

uptime [OPTION]...

Purpose

The uptime command shows how long the system has been running, the number of currently logged-in users, and the system load averages for the past 1, 5, and 15 minutes. Load averages above the CPU count indicate system overload.

Syntax

```
uptime [OPTION]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-p`	Pretty format: show uptime in human-readable form
`-s`	Show since when the system has been up (start time)

Examples

1. **`$ uptime`**
   Show uptime, users, and load averages

2. **`$ uptime -p`**
   Pretty format: 'up 2 days, 3 hours'

3. **`$ uptime -s`**
   Show system start time

4. **`$ watch -n 5 uptime`**
   Monitor load averages continuously

5. **`$ cat /proc/uptime`**
   Raw uptime in seconds from kernel

\$w

w [OPTION]... [USER]

Purpose

The w command shows who is currently logged on and what they are doing. It displays user login time, terminal, remote host, idle time, CPU time used, and the current command running. It combines the functionality of who and uptime.

Syntax

```
w [OPTION]... [USER]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-h`	Omit header line
`-s`	Short format: skip idle and CPU time columns
`-f`	Toggle display of FROM field (remote host)
`-i`	Show idle time for user

Examples

1. `$ w`
   Show all logged-in users and activity
2. `$ w alice`
   Show activity for user alice
3. `$ w -h`
   Output without header
4. `$ w -s`
   Short format output
5. `$ w | awk 'NR>2{print $1}'`
   Extract just usernames

\$who

who [OPTION]... [FILE]

Purpose

The who command displays information about users who are currently logged in, showing username, terminal line, login time, and optionally the remote host. It reads from /var/run/utmp and provides a simpler output than the w command.

Syntax

```
who [OPTION]... [FILE]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-a`	All info: processes, clock, uname, run level
`-b`	Time of last system boot
`-H`	Print header/column labels
`-q`	Count only: print usernames and count
`-r`	Show current run level
`-u`	Show idle time for each user

Examples

1. **`$ who`**
   List logged-in users

2. **`$ who -b`**
   Show last system boot time

3. **`$ who am i`**
   Show your own login info

4. **`$ who -H`**
   Show with column headers

5. **`$ who -q`**
   Count logged-in users

\$whois

whois [OPTION]... QUERY

Purpose

The whois command queries the WHOIS protocol databases to retrieve registration information for domain names and IP addresses. It shows ownership, registrar, registration dates, nameservers, and contact information for internet resources.

Syntax

```
whois [OPTION]... QUERY
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-h HOST`	Use specified whois server
`-p PORT`	Connect to specified port
`-H`	Suppress legal disclaimers
`-I`	Use the IANA database

Examples

1. `$ whois example.com`
   Get domain registration info
2. `$ whois 8.8.8.8`
   Get IP address ownership info
3. `$ whois -h whois.arin.net 192.168.1.1`
   Query specific whois server
4. `$ whois github.com | grep 'Creation Date'`
   Find domain creation date
5. `$ whois google.com | grep 'Name Server'`
   Find nameservers

\$whoami

whoami

Purpose

The whoami command prints the effective username of the current user. It is useful in scripts to determine which user is running the script, verify privilege escalation (e.g., checking if running as root), and display the current identity.

Syntax

```
whoami
```

Examples

1. `$ whoami`
   Print current username
2. `$ if [ "$(whoami)" = 'root' ]; then echo 'root'; fi`
   Check if running as root
3. `$ echo "Running as: $(whoami)"`
   Embed username in message
4. `$ sudo whoami`
   Confirm sudo gives root identity
5. `$ whoami; hostname`
   Show user and hostname together

\$iostat

iostat [OPTION]... [DEVICE] [INTERVAL [COUNT]]

Purpose

The iostat command reports CPU utilization and I/O statistics for devices and partitions. It helps diagnose performance bottlenecks by showing disk read/write rates, transfer speeds, and CPU wait time, which is time spent waiting for I/O operations.

Syntax

```
iostat [OPTION]... [DEVICE] [INTERVAL [COUNT]]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-d`	Display device utilization report only
`-c`	Display CPU utilization report only
`-x`	Extended statistics (more detail per device)
`-m`	Display statistics in megabytes/second
`-h`	Human-readable output
`-k`	Display statistics in kilobytes/second
`INTERVAL`	Time between reports in seconds

Examples

1. **`$ iostat`**
   Show CPU and basic I/O stats
2. **`$ iostat -x`**
   Extended stats with await and utilization
3. **`$ iostat -d -m 2 5`**
   Disk stats in MB/s, 5 times every 2s
4. **`$ iostat -x /dev/sda`**
   Extended stats for sda only
5. **`$ iostat 1`**
   Update every second continuously

\$uname

uname [OPTION]...

Purpose

The uname command prints system information about the operating system kernel, hardware architecture, hostname, and OS version. It is commonly used in scripts to detect the operating environment and adapt behavior accordingly.

Syntax

```
uname [OPTION]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-a`	Print all information
`-s`	Kernel name (e.g., Linux)
`-r`	Kernel release version
`-v`	Kernel version
`-m`	Machine hardware name (e.g., x86_64)
`-n`	Network node hostname
`-o`	Operating system

Examples

1. `$ uname -a`
   Print all system info
2. `$ uname -r`
   Print kernel version
3. `$ uname -m`
   Print CPU architecture
4. `$ uname -s`
   Print OS name
5. `$ uname -n`
   Print hostname

\$useradd

useradd [OPTION]... USERNAME

Purpose

The useradd command creates a new user account on the system. It creates entries in /etc/passwd and /etc/shadow, optionally creates a home directory, sets the user's shell, assigns a UID, and adds the user to specified groups.

Syntax

```
useradd [OPTION]... USERNAME
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-m`	Create user's home directory
`-d DIR`	Set home directory to DIR
`-s SHELL`	Set login shell
`-g GROUP`	Set primary group
`-G GROUPS`	Add to supplementary groups (comma-separated)
`-u UID`	Specify user ID
`-c COMMENT`	Set GECOS comment field
`-e DATE`	Set account expiry date

Examples

1. `$ useradd alice`
   Create user alice with defaults
2. `$ useradd -m -s /bin/bash alice`
   Create alice with home dir and bash shell
3. `$ useradd -m -G sudo,docker alice`
   Create alice with sudo and docker groups
4. `$ useradd -u 1500 -g staff alice`
   Create alice with specific UID and group
5. `$ useradd -e 2026-12-31 tempuser`
   Create account expiring end of 2026

\$usermod

usermod [OPTION]... USERNAME

Purpose

The usermod command modifies an existing user account. It can change the username, home directory, shell, groups, UID, expiry date, and other account properties without deleting and recreating the account.

Syntax

```
usermod [OPTION]... USERNAME
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-aG GROUPS`	Append user to supplementary groups
`-s SHELL`	Change login shell
`-d DIR`	Change home directory
`-l NEWNAME`	Change login name
`-L`	Lock user account (disable login)
`-U`	Unlock user account
`-e DATE`	Set account expiry date
`-u UID`	Change user ID

Examples

1. `$ usermod -aG docker alice`
   Add alice to docker group
2. `$ usermod -s /bin/zsh alice`
   Change alice's shell to zsh
3. `$ usermod -L alice`
   Lock alice's account
4. `$ usermod -U alice`
   Unlock alice's account
5. `$ usermod -aG sudo alice`
   Give alice sudo privileges

\$vmstat

vmstat [OPTION]... [DELAY [COUNT]]

Purpose

The vmstat command reports information about processes, memory, paging, block I/O, traps, and CPU activity. It provides a compact overview of system performance, helping identify bottlenecks in memory, CPU, or disk I/O.

Syntax

```
vmstat [OPTION]... [DELAY [COUNT]]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-a`	Show active/inactive memory
`-d`	Report disk statistics
`-s`	Display memory statistics table
`-m`	Display slabinfo
`-n`	Only display header once
`DELAY`	Seconds between updates
`COUNT`	Number of updates to display

Examples

1. **`$ vmstat`**
   Single snapshot of system stats
2. **`$ vmstat 2`**
   Update every 2 seconds
3. **`$ vmstat 1 10`**
   10 updates every 1 second
4. **`$ vmstat -s`**
   Display memory statistics table
5. **`$ vmstat -d`**
   Display disk I/O statistics

\$shutdown

shutdown [OPTION]... TIME [WALL_MESSAGE]

Purpose

The shutdown command safely brings the system down, notifying logged-in users and stopping processes in an orderly manner. It can schedule shutdowns, immediate halts, reboots, and cancellations, and is the proper way to power down a Linux system.

Syntax

```
shutdown [OPTION]... TIME [WALL_MESSAGE]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-h`	Halt or power off after shutdown
`-r`	Reboot after shutdown
`-c`	Cancel a pending shutdown
`-k`	Send warning messages but do not actually shut down
`now`	Shut down immediately
`HH:MM`	Schedule shutdown at specified time
`+N`	Shut down in N minutes

Examples

1. `$ shutdown now`
   Immediate system shutdown
2. `$ shutdown -r now`
   Reboot immediately
3. `$ shutdown -h +10`
   Power off in 10 minutes
4. `$ shutdown -h 22:00 'Maintenance'`
   Schedule shutdown at 10PM
5. `$ shutdown -c`
   Cancel scheduled shutdown

\$service

service NAME COMMAND

Purpose

The service command runs System V init scripts or controls systemd services, providing a unified interface to start, stop, restart, reload, and check status of system services. It is compatible with both SysV init and systemd systems.

Syntax

```
service NAME COMMAND
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`start`	Start the named service
`stop`	Stop the named service
`restart`	Stop and then start the service
`reload`	Reload configuration without restart
`status`	Show current status of service
`enable`	Enable service to start at boot
`disable`	Disable service from starting at boot

Examples

1. `$ service nginx start`
   Start nginx web server
2. `$ service nginx restart`
   Restart nginx
3. `$ service mysql status`
   Check MySQL service status
4. `$ service --status-all`
   Show status of all services
5. `$ service ssh reload`
   Reload SSH configuration

\$userdel

userdel [OPTION]... USERNAME

Purpose

The userdel command deletes a user account from the system. It removes entries from /etc/passwd and /etc/shadow. With the -r flag, it also removes the user's home directory and mail spool, which otherwise remain on the system.

Syntax

```
userdel [OPTION]... USERNAME
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-r`	Remove home directory and mail spool
`-f`	Force removal even if user is logged in

Examples

1. `$ userdel alice`
   Remove user alice (keep home dir)
2. `$ userdel -r alice`
   Remove alice and her home directory
3. `$ userdel -f bob`
   Force remove even if logged in
4. `$ userdel -r tempuser`
   Remove temporary user and files
5. `$ getent passwd alice`
   Verify alice is removed

\$pstree

pstree [OPTION]... [PID or USER]

Purpose

The pstree command displays running processes as a tree, showing parent-child relationships between processes. It helps understand process hierarchies, identify which processes spawned others, and visualize the relationship between daemons and their children.

Syntax

```
pstree [OPTION]... [PID or USER]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-p`	Show PIDs next to process names
`-u`	Show UIDs for process owners
`-a`	Show command-line arguments
`-h`	Highlight current process and its ancestors
`-n`	Sort by PID
`-l`	Use long output lines

Examples

1. `$ pstree`
   Show process tree for all processes
2. `$ pstree -p`
   Show tree with PIDs
3. `$ pstree alice`
   Show process tree for user alice
4. `$ pstree -up`
   Show tree with PIDs and users
5. `$ pstree 1`
   Show tree starting from PID 1 (init)

\$ncdu

ncdu [OPTION]... [DIRECTORY]

Purpose

The ncdu (NCurses Disk Usage) command provides an interactive, curses-based terminal interface for exploring disk space usage. It recursively scans a directory and allows navigation through the directory tree to identify which files and directories consume the most space.

Syntax

```
ncdu [OPTION]... [DIRECTORY]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-x`	Do not cross filesystem boundaries
`-q`	Quiet mode: fewer updates while scanning
`-e`	Enable extended information
`-r`	Read-only mode: disabling deletion from UI
`--exclude PATTERN`	Exclude files matching PATTERN

Examples

1. `$ ncdu`
   Scan current directory interactively
2. `$ ncdu /`
   Scan entire filesystem
3. `$ ncdu -x /`
   Scan root without crossing mounts
4. `$ ncdu /var/log`
   Analyze log directory space
5. `$ ncdu --exclude '*.iso' /home`
   Exclude ISO files from scan

\$hostname

hostname [OPTION]... [NAME]

Purpose

The hostname command shows or sets the system's network hostname. The hostname identifies the machine on a network and is used in shell prompts, system logs, and network services. Changes made without persistence revert on reboot.

Syntax

```
hostname [OPTION]... [NAME]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-f / --fqdn`	Display the fully qualified domain name
`-i`	Display IP address(es) of the hostname
`-d`	Display DNS domain name
`-s`	Display short hostname only
`-A`	Display all FQDNs of the machine

Examples

1. `$ hostname`
   Display current hostname
2. `$ hostname newserver`
   Temporarily set hostname to newserver
3. `$ hostname -f`
   Show fully qualified domain name
4. `$ hostname -i`
   Show IP address of hostname
5. `$ hostnamectl set-hostname prod-server`
   Permanently set hostname (systemd)

\$etcdctl

etcdctl [OPTION]... COMMAND

Purpose

The etcdctl command is the CLI tool for etcd, a distributed key-value store widely used for configuration data and service discovery in Kubernetes and microservices. It allows reading, writing, watching, and managing keys in an etcd cluster.

Syntax

```
etcdctl [OPTION]... COMMAND
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`get KEY`	Get value of a key
`put KEY VALUE`	Set a key-value pair
`del KEY`	Delete a key
`watch KEY`	Watch a key for changes
`--endpoints=URL`	etcd cluster endpoints
`--user=USER:PASS`	Authentication credentials

Examples

1. `$ etcdctl put /config/db_host '10.0.0.1'`
   Store config value
2. `$ etcdctl get /config/db_host`
   Retrieve stored config
3. `$ etcdctl del /config/old_key`
   Delete a key
4. `$ etcdctl watch /config/`
   Watch for changes under /config/
5. `$ etcdctl get --prefix /config/`
   Get all keys with /config/ prefix

\$groupadd

groupadd [OPTION]... GROUP

Purpose

The groupadd command creates a new group account on the system. It adds an entry to /etc/group and /etc/gshadow. Groups are used to organize users and control access to files and system resources through group permissions.

Syntax

```
groupadd [OPTION]... GROUP
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-g GID`	Specify group ID
`-r`	Create a system group
`-f`	Force: exit without error if group exists
`-K KEY=VAL`	Override defaults from /etc/login.defs

Examples

1. `$ groupadd developers`
   Create group named developers
2. `$ groupadd -g 2000 devops`
   Create group with specific GID
3. `$ groupadd -r docker`
   Create system group docker
4. `$ groupadd -f staff`
   Create group (no error if exists)
5. `$ groupadd dbadmins`
   Create database admins group

\$access

test [OPTION]... FILE | [OPTION FILE]

Purpose

The access function (and test command) checks whether a file exists and is accessible with specified permissions for the current process. In scripts, the test command or [ ] with -r, -w, -x flags provides access checking commonly used before file operations.

Syntax

```
test [OPTION]... FILE | [ OPTION FILE ]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-r FILE`	Check if file is readable
`-w FILE`	Check if file is writable
`-x FILE`	Check if file is executable
`-e FILE`	Check if file exists
`-f FILE`	Check if file is a regular file
`-d FILE`	Check if path is a directory

Examples

1. `$ [ -r file.txt ] && cat file.txt`
   Read only if readable
2. `$ [ -w /etc ] && echo 'writable'`
   Check if /etc is writable
3. `$ [ -x ./script.sh ] && ./script.sh`
   Run only if executable
4. `$ [ -e /var/run/app.pid ] && echo 'running'`
   Check PID file existence
5. `$ [ -d /backup ] || mkdir /backup`
   Create dir if it doesn't exist

\$kill

kill [OPTION]... PID...

Purpose

The kill command sends signals to processes by PID, allowing controlled termination or interaction. The default signal is SIGTERM (15), requesting graceful shutdown. SIGKILL (9) forces immediate termination. Other signals pause, continue, or reload processes.

Syntax

```
kill [OPTION]... PID...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-9 / -SIGKILL`	Force kill: cannot be caught or ignored
`-15 / -SIGTERM`	Graceful termination (default signal)
`-1 / -SIGHUP`	Hangup: reload config for many daemons
`-19 / -SIGSTOP`	Pause process execution
`-18 / -SIGCONT`	Continue a paused process
`-l`	List all available signals

Examples

1. `$ kill 1234`
   Send SIGTERM to process 1234
2. `$ kill -9 1234`
   Force kill process 1234
3. `$ kill -HUP $(cat /var/run/nginx.pid)`
   Reload nginx config
4. `$ kill -STOP 1234`
   Pause process 1234
5. `$ kill $(pgrep -f myapp)`
   Kill process by name

\$bg

bg [JOB_SPEC...]

Purpose

The bg command resumes a suspended job in the background, allowing it to run without occupying the terminal. Jobs are suspended with Ctrl+Z and managed with the shell's job control facilities. Background jobs continue running even while you use the terminal for other work.

Syntax

```
bg [JOB_SPEC...]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
JOB_SPEC	Job number prefixed with % (e.g., %1, %2); defaults to current job

Examples

1. **`$ bg`**
   Resume current suspended job in background
2. **`$ bg %1`**
   Resume job number 1 in background
3. **`$ bg %2`**
   Resume job number 2 in background
4. **`$ sleep 100; ctrl+z; bg`**
   Suspend then background a sleep
5. **`$ jobs; bg %1`**
   List jobs then background job 1

\$fg

fg [JOB_SPEC]

Purpose

The fg command brings a background or suspended job to the foreground, connecting it to the terminal's standard input and output. It is used with shell job control to switch between multiple processes running in the same terminal session.

Syntax

```
fg [JOB_SPEC]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
JOB_SPEC	Job number prefixed with % (e.g., %1); defaults to current job

Examples

1. `$ fg`
   Bring current job to foreground
2. `$ fg %1`
   Bring job 1 to foreground
3. `$ fg %2`
   Bring job 2 to foreground
4. `$ vim file.txt; ctrl+z; fg`
   Suspend and return to vim
5. `$ jobs; fg %1`
   List jobs then foreground job 1

\$jobs

jobs [OPTION]... [JOB_SPEC]

Purpose

The jobs command lists all active jobs in the current shell session, showing their job numbers, states (Running, Stopped, Done), and commands. It is part of shell job control and is used to manage multiple processes in a single terminal session.

Syntax

```
jobs [OPTION]... [JOB_SPEC]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-l`	Long: show PID in addition to job info
`-p`	Show only PIDs of jobs
`-r`	Show only running jobs
`-s`	Show only stopped (suspended) jobs

Examples

1. **`$ jobs`**
   List all jobs in current shell
2. **`$ jobs -l`**
   List jobs with PIDs
3. **`$ jobs -r`**
   Show only running jobs
4. **`$ jobs -s`**
   Show only stopped jobs
5. **`$ sleep 100 & sleep 200 & jobs`**
   Start background jobs and list them

\$top

top [OPTION]...

Purpose

The top command provides a dynamic real-time view of running processes, displaying CPU and memory usage, process priority, running time, and more. It is the primary interactive tool for monitoring system performance and identifying resource-intensive processes.

Syntax

```
top [OPTION]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-d SEC`	Delay between screen updates in seconds
`-n N`	Update N times then exit
`-p PID`	Monitor only specific PIDs
`-u USER`	Show only processes for specific user
`-b`	Batch mode: output to file or pipe
`-H`	Show individual threads

Examples

1. `$ top`
   Interactive process monitor
2. `$ top -u alice`
   Monitor only alice's processes
3. `$ top -d 1`
   Update every 1 second
4. `$ top -n 5 -b > top_snapshot.txt`
   Save 5 snapshots to file
5. `$ top -p 1234,5678`
   Monitor specific processes

\$killall

killall [OPTION]... NAME...

Purpose

The killall command sends a signal to all processes running a specified command by name. Unlike kill which requires a PID, killall matches by process name, making it convenient to stop all instances of a named program at once.

Syntax

```
killall [OPTION]... NAME...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-9`	Force kill (SIGKILL)
`-u USER`	Kill only processes owned by USER
`-i`	Interactive: confirm before killing
`-v`	Verbose: report if signal sent successfully
`-w`	Wait until process terminates
`-r`	Interpret NAME as regex

Examples

1. `$ killall firefox`
   Kill all Firefox processes
2. `$ killall -9 java`
   Force kill all Java processes
3. `$ killall -u alice python`
   Kill alice's Python processes
4. `$ killall -i myapp`
   Interactively kill myapp
5. `$ killall -w nginx; nginx`
   Restart nginx properly

\$fuser

fuser [OPTION]... FILE...

Purpose

The fuser command identifies processes that are using specified files, directories, or sockets. It is useful for finding which process has a file open (preventing unmounting), identifying who is using a network port, and killing those processes.

Syntax

```
fuser [OPTION]... FILE...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-k`	Kill processes accessing the file
`-m`	Name a mounted filesystem
`-n SPACE`	Search in different namespaces: file, udp, tcp
`-u`	Append username of process owner
`-v`	Verbose: display additional info
`-i`	Interactive: confirm before killing

Examples

1. `$ fuser /var/log/app.log`
   Find PID using app.log
2. `$ fuser -v 80/tcp`
   Find process on TCP port 80
3. `$ fuser -km /mnt/usb`
   Kill processes and unmount safely
4. `$ fuser -u /etc/passwd`
   Show PID and user of processes
5. `$ fuser 443/tcp`
   Find process on HTTPS port

\$pkill

pkill [OPTION]... PATTERN

Purpose

The pkill command sends signals to processes based on name, user, parent PID, or other attributes, combining process lookup and signaling in one command. It is more flexible than killall and uses the same matching as pgrep.

Syntax

```
pkill [OPTION]... PATTERN
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-9`	Force kill (SIGKILL)
`-u USER`	Kill only processes owned by USER
`-P PID`	Kill only children of specified parent
`-f`	Match against full command line
`-n`	Kill only the newest matching process
`-o`	Kill only the oldest matching process
`-x`	Exact match (not substring)

Examples

1. `$ pkill firefox`
   Kill all Firefox processes
2. `$ pkill -9 -u alice`
   Force kill all of alice's processes
3. `$ pkill -f 'python script.py'`
   Kill by full command line
4. `$ pkill -P 1234`
   Kill all children of process 1234
5. `$ pkill -x sshd`
   Kill only exact matches to 'sshd'

\$wait

wait [JOB_SPEC | PID...]

Purpose

The wait command waits for background jobs or processes to complete and returns their exit status. It is essential in shell scripts for synchronization, ensuring that all background tasks finish before the script continues to the next step.

Syntax

```
wait [JOB_SPEC | PID...]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-n`	Wait for any single job to complete
`-f`	Wait for each job to terminate, not just change status

Examples

1. `$ wait`
   Wait for all background jobs to finish
2. `$ wait $PID`
   Wait for specific PID to finish
3. `$ wait %1`
   Wait for job number 1 to finish
4. `$ cmd1 & cmd2 & wait; echo 'done'`
   Run in parallel, wait for both
5. `$ PID=$!; do_other_work; wait $PID`
   Launch job, do work, then join

\$cat

cat [OPTION]... [FILE]...

Purpose

The cat command concatenates and displays file contents to standard output. It is one of the most frequently used Linux commands for viewing files, combining files, creating files from standard input, and piping data to other commands in pipelines.

Syntax

```
cat [OPTION]... [FILE]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-n`	Number all output lines
`-b`	Number non-blank lines only
`-s`	Squeeze multiple blank lines into one
`-v`	Show non-printing characters
`-E`	Show $ at end of each line
`-T`	Show tab characters as ^I
`-A`	Equivalent to -vET

Examples

1. `$ cat file.txt`
   Display file contents
2. `$ cat file1.txt file2.txt > merged.txt`
   Concatenate two files
3. `$ cat -n script.py`
   Display script with line numbers
4. `$ cat > newfile.txt`
   Create file from keyboard input (Ctrl+D to end)
5. `$ cat /dev/null > file.txt`
   Empty a file's contents

\$fold

fold [OPTION]... [FILE]...

Purpose

The fold command wraps each line of text to fit within a specified width, breaking long lines at the given column width. It is useful for formatting text output for terminals, preparing text for emails, and ensuring text fits within display constraints.

Syntax

```
fold [OPTION]... [FILE]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-w N`	Wrap at N characters (default 80)
`-s`	Break at spaces instead of in the middle of words
`-b`	Count bytes instead of columns

Examples

1. `$ fold -w 72 long_text.txt`
   Wrap at 72 characters per line
2. `$ fold -s -w 80 paragraph.txt`
   Wrap at word boundaries, max 80
3. `$ echo 'long string...' | fold -w 40`
   Fold pipeline output
4. `$ fold -w 60 -s readme.txt`
   Format readme for 60-char width
5. `$ cat file | fold -w 78 | mail user@host`
   Format for email

\$head

head [OPTION]... [FILE]...

Purpose

The head command outputs the first part of files, by default the first 10 lines. It is commonly used to preview file contents, display file headers, check log file beginnings, and extract the top of sorted outputs in pipelines.

Syntax

```
head [OPTION]... [FILE]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-n N`	Print first N lines (default 10)
`-c N`	Print first N bytes
`-q`	Never print filename headers
`-v`	Always print filename headers
`-n -N`	Print all but last N lines

Examples

1. `$ head file.txt`
   Show first 10 lines
2. `$ head -n 20 log.txt`
   Show first 20 lines
3. `$ head -c 100 binary.bin`
   Show first 100 bytes
4. `$ head -n 1 *.csv`
   Show first line (header) of each CSV
5. `$ ls -lt | head -5`
   Show 5 most recently modified files

\$ lpr

lpr [OPTION]... [FILE]...

Purpose

The lpr command submits files to the print queue for printing. It sends files to the default printer or a specified printer, with options for the number of copies, page ranges, and job priority. It is the standard Unix command for initiating print jobs.

Syntax

```
lpr [OPTION]... [FILE]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-P PRINTER`	Send to specified printer
`-# N`	Print N copies
`-J TITLE`	Set the job title
`-m`	Send email notification when done
`-h`	Suppress banner page

Examples

1. `$ lpr document.pdf`
   Print PDF on default printer
2. `$ lpr -P HP_LaserJet report.pdf`
   Print on specific printer
3. `$ lpr -# 3 flyer.pdf`
   Print 3 copies
4. `$ lpr -J 'Monthly Report' report.pdf`
   Print with job title
5. `$ cat file.txt | lpr`
   Print piped input

\$ lprm

lprm [OPTION]... [JOB_ID | -]

Purpose

The lprm command removes (cancels) print jobs from the print queue. It can remove a specific job by number, cancel all jobs for the current user, or when run as root, cancel any job in the queue. It is the companion command to lpr and lpq.

Syntax

```
lprm [OPTION]... [JOB_ID | -]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-P PRINTER`	Specify printer queue to remove from
`-`	Remove all jobs owned by current user
`JOB_ID`	Cancel specific job by number

Examples

1. `$ lprm`
   Cancel current user's most recent job
2. `$ lprm -`
   Cancel all your print jobs
3. `$ lprm 42`
   Cancel print job number 42
4. `$ lprm -P HP_Laser -`
   Cancel all jobs on specific printer
5. `$ lpq; lprm 15`
   Check queue then cancel job 15

\$more

more [OPTION]... [FILE]...

Purpose

The more command is a terminal pager that displays text one screenful at a time, pausing for user input to continue. It is one of the oldest Unix pagers; less is generally preferred as it supports backward navigation, but more is universally available.

Syntax

```
more [OPTION]... [FILE]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-N`	Display line numbers
`-d`	Display help instead of bell on error
`-f`	Count logical lines rather than screen lines
`-p`	Clear screen before displaying
`-c`	Do not scroll, draw from top
`-s`	Squeeze multiple blank lines
`-N`	Number lines of output

Examples

1. `$ more /var/log/syslog`
   View log file page by page
2. `$ cat file.txt | more`
   Page through command output
3. `$ more -d file.txt`
   Show with help prompts
4. `$ more +50 file.txt`
   Start display at line 50
5. `$ more -5 file.txt`
   Show 5 lines per screenful

\$ less

less [OPTION]... [FILE]...

Purpose

The less command is an improved terminal pager for viewing text files, supporting forward and backward navigation, search, and many more features than more. It does not read the entire file before showing content, making it fast for large files.

Syntax

```
less [OPTION]... [FILE]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-N`	Show line numbers
`-S`	Chop long lines instead of wrapping
`-i`	Case-insensitive search
`-F`	Quit automatically for short files
`-R`	Display ANSI color codes
`-X`	Do not clear screen on exit
`-G`	Do not highlight search matches

Examples

1. `$ less /var/log/syslog`
   View log file with navigation
2. `$ less -N file.txt`
   View with line numbers
3. `$ less +G file.txt`
   Start at end of file
4. `$ grep 'error' log | less`
   Page through filtered results
5. `$ less -S wide_table.txt`
   View without line wrapping

\$page

page [FILE]...

Purpose

The page command is an older pager utility similar to more, displaying text one screen at a time. It is less feature-rich than less and largely deprecated in favor of more or less, but may still be present in some Unix systems for compatibility.

Syntax

```
page [FILE]...
```

Examples

1. `$ page file.txt`
   Display file page by page
2. `$ cat output | page`
   Page through command output
3. `$ page -d file.txt`
   Display with prompts (if supported)
4. `$ page +100 file.txt`
   Start at line 100
5. `$ page large_log.txt`
   View large log interactively

\$pr

pr [OPTION]... [FILE]...

Purpose

The pr command formats and paginates text files for printing or display. It adds headers with filename, date, and page numbers, can format output into multiple columns, and merges files side by side — making it useful for preparing reports and listings.

Syntax

```
pr [OPTION]... [FILE]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-n`	Number lines
`-c N`	Format into N columns
`-l N`	Set page length to N lines
`-h HEADER`	Use HEADER as page header
`-d`	Double space output
`-m`	Merge files, one per column
`-t`	Omit page headers and trailers

Examples

1. `$ pr file.txt`
   Format file with header and page numbers
2. `$ pr -2 file.txt`
   Format into 2 columns
3. `$ pr -h 'Report' file.txt`
   Use custom header
4. `$ pr -m file1.txt file2.txt`
   Merge two files side by side
5. `$ pr -t file.txt`
   Format without headers

\$tail

tail [OPTION]... [FILE]...

Purpose

The tail command outputs the last part of files, by default the last 10 lines. Its most powerful feature is the -f (follow) option which continuously monitors a file for new content, making it the standard tool for watching log files in real time.

Syntax

```
tail [OPTION]... [FILE]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-n N`	Print last N lines
`-c N`	Print last N bytes
`-f`	Follow: keep reading as file grows
`-F`	Follow by name: reopen if renamed
`--pid=PID`	Stop following when PID terminates
`-q`	Never print filename headers
`-v`	Always print filename headers

Examples

1. `$ tail /var/log/syslog`
   Show last 10 lines of syslog
2. `$ tail -n 50 error.log`
   Show last 50 lines
3. `$ tail -f /var/log/nginx/access.log`
   Live monitoring of access log
4. `$ tail -f -n 0 app.log`
   Follow new lines only from now on
5. `$ tail -F /var/log/app.log`
   Follow log, even if it gets rotated

\$zcat

zcat [OPTION]... FILE...

Purpose

The zcat command decompresses and displays the contents of gzip-compressed files to standard output without modifying the files. It is equivalent to gunzip -c and is useful for reading compressed log files without first decompressing them to disk.

Syntax

```
zcat [OPTION]... FILE...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-f`	Force decompression even if file is not compressed
`-l`	List compressed file info
`-v`	Verbose output

Examples

1. `$ zcat log.gz`
   Display compressed log content
2. `$ zcat app.log.gz | grep 'ERROR'`
   Search compressed log for errors
3. `$ zcat file.gz | wc -l`
   Count lines in compressed file
4. `$ zcat *.log.gz | less`
   Page through multiple compressed logs
5. `$ zcat data.csv.gz > data.csv`
   Decompress to file via redirect

\$xv

xv [OPTION]... [FILE]...

Purpose

The xv command is a graphical image viewing and manipulation program for X Window systems. It supports many image formats and provides interactive tools for scaling, rotating, cropping, and color adjustment of images in a graphical environment.

Syntax

```
xv [OPTION]... [FILE]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-2`	Display at 2x magnification
`-3`	Display at 3x magnification
`-rotate N`	Rotate image N degrees
`-crop X Y W H`	Crop to specified region
`-root`	Display image as root/desktop background
`-geom WxH`	Set window geometry

Examples

1. `$ xv image.jpg`
   Open image in xv viewer
2. `$ xv -2 photo.png`
   View at 2x magnification
3. `$ xv -rotate 90 image.jpg`
   Display rotated 90 degrees
4. `$ xv -root wallpaper.jpg`
   Set as desktop background
5. `$ xv *.jpg`
   Browse all JPEG files

\$ gv

gv [OPTION]... [FILE]

Purpose

The gv command is an X11 graphical PostScript and PDF viewer built on the Ghostscript engine. It provides navigation, zoom, and display controls for PostScript (.ps) and PDF files in a graphical environment using the X Window System.

Syntax

```
gv [OPTION]... [FILE]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-scale=N`	Set initial zoom scale
`-landscape`	Display in landscape orientation
`-portrait`	Display in portrait orientation
`-spartan`	Minimal interface mode

Examples

1. `$ gv document.ps`
   View a PostScript file
2. `$ gv report.pdf`
   View a PDF document
3. `$ gv -scale=2 document.ps`
   Open at 2x zoom
4. `$ gv -landscape poster.ps`
   View in landscape mode
5. `$ gv -spartan file.pdf`
   Open with minimal UI

\$xpdf

xpdf [OPTION]... [FILE] [PAGE]

Purpose

The xpdf command is a lightweight open-source PDF viewer for X Window systems. It renders PDF files with good fidelity, supports text extraction, copy-to-clipboard, full-text search, and can be scripted via command line for automated PDF processing.

Syntax

```
xpdf [OPTION]... [FILE] [PAGE]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-z ZOOM`	Set initial zoom level
`-r N`	Set screen resolution in DPI
`-fullscreen`	Open in fullscreen mode
`-g WxH+X+Y`	Set window geometry
`-pw PASS`	Provide PDF open password

Examples

1. `$ xpdf document.pdf`
   Open PDF with xpdf viewer
2. `$ xpdf -z 150 report.pdf`
   Open at 150% zoom
3. `$ xpdf report.pdf 5`
   Open at page 5
4. `$ xpdf -fullscreen slides.pdf`
   Open in fullscreen
5. `$ xpdf -pw mypass secured.pdf`
   Open password-protected PDF

\$ logout

logout

Purpose

The logout command exits the current login shell session. In contrast to exit (which exits any shell), logout only works in a login shell. It closes the terminal session, disconnects remote SSH sessions, and cleans up the user's environment.

Syntax

```
logout
```

Examples

1. `$ logout`
   Log out of current session
2. `$ ssh server; logout`
   Connect to server and log out when done
3. `$ sudo bash -l; logout`
   Enter root login shell then exit
4. `$ exit`
   Alternative to logout in non-login shells
5. `$ trap 'echo Bye!' EXIT; logout`
   Print message before logging out

\$passwd

passwd [OPTION]... [USERNAME]

Purpose

The passwd command changes a user's login password. When run by a normal user it changes their own password, prompting for the old password first. When run by root it can change any user's password, force a password reset, lock/unlock accounts, and set expiry.

Syntax

```
passwd [OPTION]... [USERNAME]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-l`	Lock account: prepend ! to password hash
`-u`	Unlock a locked account
`-e`	Expire password: force change on next login
`-d`	Delete password (empty password — insecure)
`-S`	Display account status
`-n DAYS`	Minimum days before password can be changed
`-x DAYS`	Maximum days before password must be changed

Examples

1. `$ passwd`
   Change your own password
2. `$ passwd alice`
   Change alice's password (root only)
3. `$ passwd -l bob`
   Lock bob's account
4. `$ passwd -u bob`
   Unlock bob's account
5. `$ passwd -e alice`
   Force alice to change password at next login

\$rlogin

rlogin [OPTION]... HOST

Purpose

The rlogin command provides remote login to another Unix system using the rlogin protocol. It is a legacy insecure tool, transmitting data in plain text, and has been completely superseded by SSH. Most modern systems disable rlogin services by default.

Syntax

```
rlogin [OPTION]... HOST
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-l USER`	Login as USER on the remote host
`-e CHAR`	Set escape character (default: ~
`-8`	Allow 8-bit data path

Examples

1. `$ rlogin server.local`
   Log into server.local
2. `$ rlogin -l admin remotehost`
   Login as admin on remotehost
3. `$ rlogin -l root 192.168.1.5`
   Login as root remotely
4. `$ rlogin legacy-unix`
   Access legacy Unix system
5. `$ ssh -o 'Protocol 1' host`
   SSH as modern rlogin alternative

\$slogin

slogin [OPTION]... [USER@]HOST [COMMAND]

Purpose

The slogin command is a symbolic link or alias for the SSH (Secure Shell) client, providing a secure alternative to rlogin. It establishes encrypted remote login sessions using the SSH protocol, authenticating with passwords or public keys.

Syntax

```
slogin [OPTION]... [USER@]HOST [COMMAND]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-l USER`	Login as USER on remote host
`-i FILE`	Use identity (private key) file
`-p PORT`	Connect to specified port

Examples

1. `$ slogin server.example.com`
   Secure login to remote host
2. `$ slogin -l admin server`
   Login as admin on server
3. `$ slogin user@192.168.1.10`
   Login with user@host syntax
4. `$ slogin -p 2222 server.com`
   Connect on port 2222
5. `$ slogin -i ~/.ssh/key host`
   Use specific private key

\$emacs

emacs [OPTION]... [FILE]...

Purpose

The emacs command launches the GNU Emacs text editor, an extensible, customizable, and self-documenting editor with its own scripting language (Emacs Lisp). It is a comprehensive text editing environment supporting syntax highlighting, version control, terminals, email, and much more.

Syntax

```
emacs [OPTION]... [FILE]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-nw`	No window: run in terminal mode
`-q`	Do not load init file
`-l FILE`	Load Emacs Lisp file FILE
`--batch`	Run in batch mode (non-interactive)
`-Q`	Do not load any init files
`--eval EXPR`	Evaluate EXPR after init

Examples

1. `$ emacs file.py`
   Open Python file in Emacs
2. `$ emacs -nw config.txt`
   Open in terminal (no GUI)
3. `$ emacs -q file.txt`
   Open without loading init file
4. `$ emacs --batch -l script.el`
   Run Emacs Lisp script non-interactively
5. `$ emacs -nw -q --eval '(message "hi")'`
   Quick evaluation in terminal

\$pico

pico [OPTION]... [FILE]

Purpose

The pico command is a simple, easy-to-use terminal text editor that is part of the Pine email client package. It displays a command help bar at the bottom of the screen, making it beginner-friendly. nano is the open-source clone that replaced it on most systems.

Syntax

```
pico [OPTION]... [FILE]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-w`	Disable long line wrapping
`-m`	Enable mouse support
`-z`	Enable Ctrl+Z suspend
`-b`	Show buffer statistics in status bar

Examples

1. `$ pico file.txt`
   Open file in pico editor
2. `$ pico -w config.conf`
   Edit without line wrapping
3. `$ pico /etc/hosts`
   Edit hosts file
4. `$ nano file.txt`
   Use nano as modern pico alternative
5. `$ pico -m script.sh`
   Open with mouse support

\$sed

sed [OPTION]... 'SCRIPT' [FILE]...

Purpose

The sed command (stream editor) filters and transforms text non-interactively. It processes text line by line applying editing commands — substitution, deletion, insertion, addressing — making it powerful for text processing in shell pipelines and scripts.

Syntax

```
sed [OPTION]... 'SCRIPT' [FILE]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-n`	Suppress automatic printing of lines
`-e SCRIPT`	Execute multiple scripts
`-i[SUFFIX]`	Edit file in-place (backup with SUFFIX)
`-r / -E`	Use extended regular expressions
`-f FILE`	Read script from file

Examples

1. `$ sed 's/old/new/g' file.txt`
   Replace all 'old' with 'new'
2. `$ sed -i 's/foo/bar/g' file.txt`
   In-place replacement
3. `$ sed -n '10,20p' file.txt`
   Print lines 10 to 20
4. `$ sed '/^#/d' config.conf`
   Delete comment lines
5. `$ sed 's/[0-9]\+/NUM/g' file.txt`
   Replace all numbers with NUM

\$vim

vim [OPTION]... [FILE]...

Purpose

The vim command launches the Vi Improved text editor, a powerful modal editor supporting syntax highlighting, multiple buffers, macros, plugins, and extensive keyboard shortcuts. It is one of the most popular editors for system administration and programming.

Syntax

```
vim [OPTION]... [FILE]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-R`	Read-only mode: cannot save changes
`-c CMD`	Execute CMD after opening file
`-p`	Open files in separate tabs
`-o`	Open files in horizontal split windows
`-O`	Open files in vertical split windows
`-u VIMRC`	Use specified vimrc instead of default
`+ or +N`	Start at last line or line N

Examples

1. `$ vim file.py`
   Open Python file in vim
2. `$ vim -R /etc/passwd`
   View passwd file read-only
3. `$ vim +50 log.txt`
   Open at line 50
4. `$ vim -p file1.txt file2.txt`
   Open in tabs
5. `$ vim -c 'set number' file.txt`
   Open with line numbers enabled

\$ftp

ftp [OPTION]... [HOST [PORT]]

Purpose

The ftp command connects to FTP servers for uploading and downloading files. It provides an interactive client with commands to list, get, put, and manage files on remote FTP servers. SFTP via SSH is strongly preferred for security on untrusted networks.

Syntax

```
ftp [OPTION]... [HOST [PORT]]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-n`	No auto-login: do not attempt autologin
`-v`	Verbose: show all responses from server
`-d`	Debug: show all FTP commands sent
`-i`	Turn off interactive prompting during transfers
`-p`	Use passive mode

Examples

1. `$ ftp ftp.example.com`
   Connect to FTP server interactively
2. `$ ftp -n -v ftp.example.com`
   Verbose connect without autologin
3. `$ ftp -i ftp.site.com`
   Connect without mget/mput prompting
4. `$ lftp ftp://user:pass@host/dir`
   Modern alternative with full features
5. `$ ncftp -u user -p pass ftp.host.com`
   Non-interactive FTP alternative

\$rsync

rsync [OPTION]... SRC DEST

Purpose

The rsync command synchronizes files and directories between two locations efficiently, transferring only the differences between source and destination. It supports local and remote sync over SSH, incremental transfers, and is the standard tool for backups.

Syntax

```
rsync [OPTION]... SRC DEST
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-a`	Archive mode: recursive, preserve permissions, times, links
`-v`	Verbose: show files being transferred
`-z`	Compress data during transfer
`-P`	Show progress and keep partial files
`--delete`	Delete destination files not in source
`-n / --dry-run`	Simulate transfer without making changes
`-e ssh`	Use SSH as transport
`--exclude=PAT`	Exclude files matching PATTERN

Examples

1. `$ rsync -av /src/ /dst/`
   Sync local directories verbosely
2. `$ rsync -avz -e ssh user@host:/src/ /dst/`
   Sync from remote over SSH
3. `$ rsync -av --delete /src/ /backup/`
   Mirror with deletion of removed files
4. `$ rsync -avzP large_file user@host:/dst/`
   Transfer large file with progress
5. `$ rsync -n -av /src/ /dst/`
   Dry run: preview without copying

\$scp

scp [OPTION]... SRC DEST

Purpose

The scp command (Secure Copy) copies files between local and remote hosts using SSH encryption. It is the standard secure alternative to rcp, supporting single file copies and directory copies with authentication and encryption provided by the SSH protocol.

Syntax

```
scp [OPTION]... SRC DEST
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-r`	Recursively copy entire directories
`-P PORT`	Specify SSH port to connect to
`-i FILE`	Use identity (private key) file
`-p`	Preserve modification times and permissions
`-q`	Quiet mode: suppress progress meter
`-C`	Enable SSH compression

Examples

1. `$ scp file.txt user@host:/remote/path/`
   Copy local file to remote host
2. `$ scp user@host:/remote/file.txt ./`
   Copy remote file to local
3. `$ scp -r /local/dir user@host:/remote/`
   Copy directory to remote
4. `$ scp -P 2222 file.txt user@host:`
   Use non-standard SSH port
5. `$ scp -i ~/.ssh/key user@host:file.txt ./`
   Use specific SSH key

\$apropos

apropos [OPTION]... KEYWORD...

Purpose

The apropos command searches the man page descriptions for keywords, helping you find commands related to a topic when you don't know the exact command name. It queries the whatis database and returns commands with matching names or short descriptions.

Syntax

```
apropos [OPTION]... KEYWORD...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-a`	All: match all keywords (AND logic)
`-r`	Interpret keywords as regex
`-s SECTION`	Search specific manual section
`-l`	Do not truncate output to terminal width
`-e`	Search exactly, not as substring

Examples

1. `$ apropos 'copy file'`
   Find commands for copying files
2. `$ apropos compression`
   Find compression-related commands
3. `$ apropos -s 1 network`
   Search section 1 for network commands
4. `$ apropos -a copy recursive`
   Find commands matching both keywords
5. `$ apropos password`
   Find password management commands

\$find

find [PATH...] [EXPRESSION]

Purpose

The find command searches for files and directories in a directory tree based on various criteria including name, type, size, modification time, permissions, and owner. It can execute commands on found files, making it extremely powerful for file management and batch processing.

Syntax

```
find [PATH...] [EXPRESSION]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-name PATTERN`	Match files by name (case-sensitive)
`-iname PATTERN`	Match files by name (case-insensitive)
`-type TYPE`	f=file, d=directory, l=link, etc.
`-size N[cwbkMG]`	Match by file size
`-mtime N`	Modified N days ago
`-newer FILE`	Modified more recently than FILE
`-exec CMD {} ;`	Execute CMD for each found file
`-maxdepth N`	Descend at most N levels

Examples

1. `$ find /home -name '*.txt'`
   Find all .txt files under /home
2. `$ find . -type f -mtime -7`
   Files modified in last 7 days
3. `$ find / -size +100M`
   Find files larger than 100MB
4. `$ find . -name '*.log' -exec rm {} ;`
   Delete all log files found
5. `$ find /etc -type f -newer /etc/hostname`
   Files newer than /etc/hostname

\$info

info [OPTION]... [MENU-ITEM...]

Purpose

The info command reads documentation stored in GNU info format, which provides a more structured and hypertextual documentation system than man pages. GNU tools often have richer documentation in info format with cross-references and navigation between sections.

Syntax

```
info [OPTION]... [MENU-ITEM...]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`--usage`	Show short usage message
`-k STRING`	Search index for STRING (like apropos)
`-n NODE`	Open to specific node
`-o FILE`	Output selected nodes to FILE
`-a`	All nodes: visit subnodes automatically
`-f FILE`	Use FILE as the info file to read

Examples

1. `$ info bash`
   Read bash info documentation
2. `$ info coreutils ls`
   Read ls documentation in coreutils
3. `$ info grep`
   View full grep reference
4. `$ info -k 'copy files'`
   Search info index for 'copy files'
5. `$ info --usage`
   Show info command usage summary

\$whatis

whatis [OPTION]... NAME...

Purpose

The whatis command displays one-line manual page descriptions for commands. It queries the whatis database (built by makewhatis or mandb) and quickly tells you what a command does, making it useful when you know a command name but need a brief reminder of its purpose.

Syntax

```
whatis [OPTION]... NAME...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-s SECTION`	Search specific manual section
`-r`	Interpret NAME as regex
`-w`	Allow NAME to be a wildcard
`-l`	Do not truncate to terminal width

Examples

1. **`$ whatis ls`**
   Show one-line description of ls

2. **`$ whatis grep sed awk`**
   Describe multiple commands at once

3. **`$ whatis -s 5 passwd`**
   Describe passwd in section 5 only

4. **`$ whatis -r 'net.*'`**
   Find commands matching regex

5. **`$ whatis tar gzip zip`**
   Compare archiving commands

\$whereis

whereis [OPTION]... NAME...

Purpose

The whereis command locates the binary, source, and manual page files for a command. It searches in standard system directories and returns the paths to the executable, source code, and man page associated with the specified command name.

Syntax

```
whereis [OPTION]... NAME...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-b`	Search only for binary executables
`-m`	Search only for manual pages
`-s`	Search only for source code
`-u`	Show commands with unusual or unusual locations
`-B LIST`	Search in specified binary directories

Examples

1. `$ whereis ls`
   Find binary, source, and man pages for ls
2. `$ whereis -b python3`
   Find only the binary for python3
3. `$ whereis -m grep`
   Find only the man pages for grep
4. `$ whereis bash vim python`
   Locate multiple commands
5. `$ whereis -u find`
   Find commands with unusual entries

\$pine

pine [OPTION]... [ADDRESS]

Purpose

The pine command (Program for Internet News and Email) is a text-based email and news client developed at the University of Washington. It provides an easy-to-use interface for reading, writing, and managing email in terminal environments. Alpine is its modern open-source successor.

Syntax

```
pine [OPTION]... [ADDRESS]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-i`	Start at inbox
`-f FOLDER`	Open specified folder on startup
`-c N`	Use connection N from config
`-url URL`	Open a specific URL

Examples

1. **`$ pine`**
   Open pine email client

2. **`$ pine -i`**
   Open directly to inbox

3. **`$ pine user@example.com`**
   Compose email to address

4. **`$ pine -f INBOX`**
   Open pine at INBOX folder

5. **`$ alpine`**
   Use alpine as modern pine alternative

\$mesg

mesg [y|n]

Purpose

The mesg command controls whether other users can write messages to your terminal using the write or talk commands. Setting mesg to n prevents message interruptions during work, while y allows it. The change lasts for the current terminal session.

Syntax

```
mesg [y|n]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
y	Allow messages from other users to your terminal
n	Disallow messages (refuse write/talk)

Examples

1. `$ mesg`
   Show current message permission status
2. `$ mesg y`
   Allow messages from other users
3. `$ mesg n`
   Block messages from other users
4. `$ who -T`
   See which users allow messages (+ = yes)
5. `$ mesg n; vim file.txt`
   Block messages before editing

\$talk

talk USER [@HOST] [TERMINAL]

Purpose

The talk command establishes a real-time, split-screen chat session between two users on the same or different systems. It requests permission from the other user and creates a two-way terminal conversation window. It requires the talkd daemon to be running.

Syntax

```
talk USER [@HOST] [TERMINAL]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
USER	Username of person to contact
@HOST	Remote hostname for cross-system chat

Examples

1. `$ talk alice`
   Start chat with local user alice
2. `$ talk alice@server.com`
   Talk to alice on remote server
3. `$ talk bob pts/2`
   Contact bob on specific terminal
4. `$ mesg y`
   Ensure your messages are enabled first
5. `$ who`
   Find logged-in users to talk to

\$write

write USER [TTY]

Purpose

The write command sends a message to another logged-in user's terminal in real time. It copies lines from standard input to the recipient's terminal until EOF (Ctrl+D). The receiving user's terminal must have mesg permission set to y.

Syntax

```
write USER [TTY]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
USER	Username of recipient
TTY	Specific terminal if user has multiple sessions

Examples

1. **`$ write alice`**
   Write message to user alice
2. **`$ write alice pts/1`**
   Write to alice on specific terminal
3. **`$ echo 'Meeting in 5' | write bob`**
   Send single-line message via pipe
4. **`$ who`**
   Find who is logged in and on which tty
5. **`$ write root pts/0`**
   Write urgent message to root

\$strace

strace [OPTION]... COMMAND | -p PID

Purpose

The strace command traces system calls and signals made by a process. It intercepts and logs all interactions between a process and the Linux kernel, making it invaluable for debugging applications, diagnosing performance issues, and understanding program behavior.

Syntax

```
strace [OPTION]... COMMAND | -p PID
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-p PID`	Attach to an existing process by PID
`-o FILE`	Write output to FILE
`-e CALL`	Trace only specified system calls
`-f`	Follow child processes created by fork()
`-c`	Summary of system calls and times
`-t`	Print timestamp for each call
`-T`	Show time spent in each call

Examples

1. `$ strace ls`
   Trace all system calls made by ls
2. `$ strace -p 1234`
   Attach and trace running process
3. `$ strace -e open,read,write cat file.txt`
   Trace only open, read, write calls
4. `$ strace -c find / -name file`
   Summarize system call statistics
5. `$ strace -o trace.log -f ./myapp`
   Trace app and children to file

\$pv

pv [OPTION]... [FILE]...

Purpose

The pv command (Pipe Viewer) monitors the progress of data through a pipeline, showing throughput, elapsed time, estimated time remaining, and a progress bar. It is useful for tracking long-running data transfer operations that would otherwise give no feedback.

Syntax

```
pv [OPTION]... [FILE]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-p`	Show progress bar
`-t`	Show elapsed time
`-e`	Show estimated time remaining
`-r`	Show current transfer rate
`-b`	Show total bytes transferred
`-s SIZE`	Estimated total size for progress percentage
`-n`	Numeric output mode (for scripts)
`-q`	Quiet: no progress output

Examples

1. `$ pv file.iso > /dev/sdb`
   Monitor ISO write to USB drive
2. `$ cat large.sql | pv | mysql db`
   Track database import progress
3. `$ tar -czf - /data | pv > backup.tar.gz`
   Monitor backup creation
4. `$ pv -s 2G large_file | gzip > compressed.gz`
   Show progress with known size
5. `$ dd if=/dev/sda | pv | dd of=/dev/sdb`
   Monitor disk-to-disk copy

\$groupdel

groupdel [OPTION]... GROUP

Purpose

The groupdel command deletes a group from the system. It removes the group entry from /etc/group and /etc/gshadow. Files owned by the deleted group retain the numeric GID; no users or files are automatically modified when a group is deleted.

Syntax

```
groupdel [OPTION]... GROUP
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-f`	Force deletion even if it is a primary group of a user

Examples

1. `$ groupdel oldteam`
   Delete group named oldteam
2. `$ groupdel -f tempgroup`
   Force delete group
3. `$ cat /etc/group | grep 'devops'`
   Verify group exists first
4. `$ groupdel devops && cat /etc/group`
   Delete and verify removal
5. `$ find / -group 1001 -print`
   Find files still owned by deleted GID

\$groupmod

groupmod [OPTION]... GROUP

Purpose

The groupmod command modifies an existing group definition on the system. It can rename a group or change its GID. Changes are written to /etc/group and /etc/gshadow, and file ownership based on the old GID may need to be updated separately.

Syntax

```
groupmod [OPTION]... GROUP
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-n NEW_NAME`	Rename the group to NEW_NAME
`-g NEW_GID`	Change the group ID to NEW_GID
`-o`	Allow non-unique GID (use with -g)

Examples

1. `$ groupmod -n newname oldname`
   Rename group from oldname to newname
2. `$ groupmod -g 2500 developers`
   Change GID of developers group
3. `$ groupmod -n devops engineers`
   Rename engineers group to devops
4. `$ grep devops /etc/group`
   Verify group name change
5. `$ find / -gid 1001 -exec chgrp devops {} ;`
   Update files after GID change

\$groups

groups [USERNAME]...

Purpose

The groups command displays the group memberships of a user. When called without arguments, it shows the groups of the current user. It reads from /etc/group and shows both the primary group and all supplementary groups the user belongs to.

Syntax

```
groups [USERNAME]...
```

Examples

1. **$ groups**
   Show current user's groups
2. **$ groups alice**
   Show alice's group memberships
3. **$ groups root**
   Show root's groups
4. **$ id alice**
   Show uid, gid, and groups with IDs
5. **$ cat /etc/group | grep alice**
   Search group file for alice

\$ethtool

ethtool [OPTION]... DEVNAME

Purpose

The ethtool command queries and controls Ethernet device settings, including link speed, duplex mode, auto-negotiation, ring buffer sizes, offload features, and driver information. It is used to diagnose network interface issues and optimize Ethernet performance.

Syntax

```
ethtool [OPTION]... DEVNAME
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-i`	Driver information for device
`-S`	Device statistics counters
`-t`	Run adapter self-test
`-s SPEED`	Set link speed
`-K`	Change offload features
`--show-ring`	Display ring buffer sizes
`--set-ring`	Change ring buffer sizes

Examples

1. `$ ethtool eth0`
   Show link status and settings for eth0
2. `$ ethtool -i eth0`
   Show driver info for eth0
3. `$ ethtool -S eth0`
   Show NIC statistics
4. `$ ethtool -s eth0 speed 1000 duplex full`
   Set 1Gbps full duplex
5. `$ ethtool -k eth0`
   Show offload feature status

\$apt

apt [OPTION]... COMMAND [PACKAGE]...

Purpose

The apt command is the high-level package management tool for Debian-based Linux distributions. It handles installing, updating, removing, and searching for packages, automatically resolving and fetching dependencies from configured repositories.

Syntax

```
apt [OPTION]... COMMAND [PACKAGE]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`install PKG`	Install a package
`remove PKG`	Remove a package
`purge PKG`	Remove package and its config files
`update`	Refresh package list from repositories
`upgrade`	Upgrade all installed packages
`autoremove`	Remove unneeded packages
`search TERM`	Search for packages by keyword
`show PKG`	Display package information

Examples

1. `$ apt update && apt upgrade`
   Update package list and upgrade all
2. `$ apt install nginx`
   Install nginx web server
3. `$ apt remove --purge apache2`
   Remove apache2 and its config
4. `$ apt search 'text editor'`
   Search for text editor packages
5. `$ apt show python3`
   Display python3 package details

\$rpm

rpm [OPTION]... PACKAGE...

Purpose

The rpm command manages RPM packages on Red Hat-based Linux distributions. It installs, removes, verifies, and queries RPM packages, providing low-level package management. The yum and dnf commands provide higher-level dependency resolution on top of rpm.

Syntax

```
rpm [OPTION]... PACKAGE...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-i`	Install a package
`-U`	Upgrade or install a package
`-e`	Erase (remove) a package
`-q`	Query: check if package is installed
`-qa`	Query all installed packages
`-ql PKG`	List files installed by package
`-qf FILE`	Find which package owns FILE
`-v`	Verbose output
`-h`	Show hash marks during install

Examples

1. `$ rpm -ivh package.rpm`
   Install RPM with verbose progress
2. `$ rpm -Uvh package.rpm`
   Upgrade or install package
3. `$ rpm -qa`
   List all installed packages
4. `$ rpm -qf /usr/bin/vim`
   Find which package provides vim
5. `$ rpm -ql httpd`
   List all files from httpd package

\$yum

yum [OPTION]... COMMAND [PACKAGE]...

Purpose

The yum command (Yellowdog Updater Modified) is the package manager for Red Hat/CentOS Linux systems, providing automatic dependency resolution, repository management, and group installation. It has been succeeded by dnf on newer RHEL and Fedora systems.

Syntax

```
yum [OPTION]... COMMAND [PACKAGE]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`install PKG`	Install package with dependencies
`remove PKG`	Remove package
`update PKG`	Update specific or all packages
`search TERM`	Search repositories for packages
`info PKG`	Show package information
`list installed`	List all installed packages
`groupinstall GROUP`	Install a package group

Examples

1. `$ yum update`
   Update all installed packages
2. `$ yum install httpd`
   Install Apache web server
3. `$ yum remove httpd`
   Remove Apache
4. `$ yum search database`
   Search for database packages
5. `$ yum info nginx`
   Show nginx package information

\$hostnamectl

hostnamectl [OPTION]... COMMAND

Purpose

The hostnamectl command queries and changes the system hostname and related settings in a systemd-based Linux system. It can set the static hostname (permanent), transient hostname (runtime), and pretty hostname (human-readable display name).

Syntax

```
hostnamectl [OPTION]... COMMAND
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`status`	Show current hostname and related info
`set-hostname NAME`	Set the hostname
`--static`	Set the static hostname
`--transient`	Set the transient (runtime) hostname
`--pretty`	Set the pretty hostname (can include spaces)

Examples

1. `$ hostnamectl status`
   Show hostname and OS info
2. `$ hostnamectl set-hostname webserver01`
   Set hostname permanently
3. `$ hostnamectl set-hostname 'Web Server 01' --pretty`
   Set human-readable name
4. `$ hostnamectl set-hostname temp-name --transient`
   Set temporary hostname
5. `$ hostname`
   Verify new hostname is active

\$reboot

reboot [OPTION]...

Purpose

The reboot command reboots the system, stopping all processes and restarting the machine. It is a safe, controlled way to restart a Linux system, ensuring filesystems are properly unmounted and cached data is written before the reboot occurs.

Syntax

```
reboot [OPTION]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-f`	Force reboot without calling init/systemd
`-p`	Poweroff instead of reboot
`--no-wall`	Do not send wall message before reboot
`-h`	Halt after shutdown

Examples

1. `$ reboot`
   Reboot the system immediately
2. `$ reboot -f`
   Force reboot bypassing normal shutdown
3. `$ shutdown -r now`
   Reboot using shutdown command
4. `$ shutdown -r +5 'Rebooting in 5 minutes'`
   Schedule reboot with message
5. `$ systemctl reboot`
   Reboot via systemctl

\$chsh

chsh [OPTION]... [USERNAME]

Purpose

The chsh command changes a user's login shell. A regular user can change their own shell to any listed in /etc/shells; the root user can change any user's shell to any program. The change takes effect on the next login.

Syntax

```
chsh [OPTION]... [USERNAME]
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-s SHELL`	Set the login shell to SHELL
`-l`	List all available shells from /etc/shells

Examples

1. `$ chsh -s /bin/zsh`
   Change your shell to zsh
2. `$ chsh -s /bin/bash alice`
   Change alice's shell to bash
3. `$ chsh -l`
   List available login shells
4. `$ cat /etc/shells`
   View all valid shells
5. `$ chsh -s /usr/bin/fish`
   Switch to Fish shell

\$compress

compress [OPTION]... FILE...

Purpose

The compress command reduces file size using the Lempel-Ziv compression algorithm, creating files with a .Z extension. It is an older Unix compression utility; gzip and bzip2 offer better compression ratios and are the standard on modern Linux systems.

Syntax

```
compress [OPTION]... FILE...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-d`	Decompress (same as uncompress)
`-v`	Verbose: show compression ratio
`-c`	Write to stdout; keep original file
`-f`	Force compression even if file grows

Examples

1. **`$ compress file.txt`**
   Compress file.txt to file.txt.Z

2. **`$ compress -v *.dat`**
   Compress all .dat files verbosely

3. **`$ compress -c file.txt > file.Z`**
   Compress to stdout

4. **`$ compress -d file.Z`**
   Decompress file.Z

5. **`$ uncompress file.Z`**
   Alternative decompress command

\$dpkg

dpkg [OPTION]... ACTION PACKAGE

Purpose

The dpkg command is the low-level Debian package manager for installing, removing, and providing information about .deb packages. The apt command provides higher-level functionality built on top of dpkg, handling dependencies and repositories automatically.

Syntax

```
dpkg [OPTION]... ACTION PACKAGE
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-i`	Install a .deb package file
`-r`	Remove installed package
`-P`	Purge package and all config files
`-l PATTERN`	List packages matching pattern
`-s PKG`	Show installed package status
`-L PKG`	List files installed by package
`-S FILE`	Find which package owns FILE
`--configure PKG`	Configure an unpacked package

Examples

1. `$ dpkg -i package.deb`
   Install a local .deb file
2. `$ dpkg -l`
   List all installed packages
3. `$ dpkg -l '*nginx*'`
   List nginx-related packages
4. `$ dpkg -L nginx`
   List files installed by nginx
5. `$ dpkg -S /usr/bin/vim`
   Find which package provides vim

\$dosfsck

dosfsck [OPTION]... DEVICE

Purpose

The dosfsck command checks and repairs MS-DOS FAT filesystems (FAT12, FAT16, FAT32). It verifies the integrity of FAT filesystems on USB drives, SD cards, and other removable media, fixing errors like bad allocation tables and cross-linked files.

Syntax

```
dosfsck [OPTION]... DEVICE
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-a`	Automatic repair: no questions asked
`-r`	Interactive repair: ask before each fix
`-n`	No-op mode: just check, do not repair
`-v`	Verbose output
`-V`	Verify: re-read after writing
`-t`	Mark unreadable clusters as bad

Examples

1. `$ dosfsck /dev/sdb1`
   Check FAT filesystem on USB partition
2. `$ dosfsck -a /dev/sdc1`
   Auto-repair FAT filesystem
3. `$ dosfsck -n /dev/sdb1`
   Check without repairing
4. `$ fsck.vfat -a /dev/sdb1`
   Alternative command (same tool)
5. `$ dosfsck -v /dev/sdc`
   Verbose check with details

\$ftpd

ftpd [OPTION]...

Purpose

The ftpd command is the FTP server daemon that handles incoming FTP client connections. It accepts connections, authenticates users, and manages file transfer operations. It should be replaced by SFTP (via SSH) in modern environments for security reasons.

Syntax

```
ftpd [OPTION]...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-d`	Debug: log FTP commands to syslog
`-l`	Log all file transfers to syslog
`-u UMASK`	Set umask for uploaded files
`-t TIMEOUT`	Set connection timeout in seconds
`-a`	Use anonymous login only

Examples

1. `$ systemctl start vsftpd`
   Start the vsftpd FTP server
2. `$ systemctl enable vsftpd`
   Enable vsftpd on boot
3. `$ netstat -tlnp | grep :21`
   Verify FTP is listening on port 21
4. `$ journalctl -u vsftpd`
   View vsftpd logs
5. `$ systemctl status vsftpd`
   Check vsftpd service status

\$gpasswd

gpasswd [OPTION]... GROUP

Purpose

The gpasswd command administers /etc/group and /etc/gshadow. It manages group passwords, adds and removes group members, and sets group administrators. It allows restricted access to groups via passwords for non-members.

Syntax

```
gpasswd [OPTION]... GROUP
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-a USER`	Add USER to GROUP
`-d USER`	Remove USER from GROUP
`-M USER1,USER2`	Set group members list
`-A USER1,USER2`	Set group administrators
`-r`	Remove group password
`-R`	Restrict access to group password

Examples

1. `$ gpasswd -a alice docker`
   Add alice to docker group
2. `$ gpasswd -d bob sudo`
   Remove bob from sudo group
3. `$ gpasswd -M alice,bob,carol developers`
   Set exact member list
4. `$ gpasswd -A alice developers`
   Make alice a group admin
5. `$ gpasswd -r staff`
   Remove group password

\$hexdump

hexdump [OPTION]... FILE...

Purpose

The hexdump command displays the contents of a file in hexadecimal, decimal, octal, or ASCII format. It is essential for analyzing binary files, debugging binary data formats, viewing file magic bytes, and inspecting network packets or encoded data.

Syntax

```
hexdump [OPTION]... FILE...
```

Common Parameters

OPTION / FLAG	DESCRIPTION
`-C`	Canonical: hex + ASCII side by side (most readable)
`-n N`	Interpret only N bytes of input
`-s N`	Skip N bytes from beginning
`-v`	Verbose: do not replace duplicate lines with *
`-e FORMAT`	Use specified output format string
`-d`	Two-byte decimal display

Examples

1. **`$ hexdump -C file.bin`**
   Hex+ASCII dump of binary file

2. **`$ hexdump -C -n 16 file.bin`**
   Show only first 16 bytes

3. **`$ hexdump -C image.jpg | head -4`**
   View JPEG magic bytes

4. **`$ hexdump -s 512 -n 512 disk.img`**
   Read 512 bytes starting at offset 512

5. **`$ xxd file.bin`**
   Alternative hex dump with xxd